



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

FPGA Enhancements in Speed, Memory, and Control for a High-Precision Arbitrary Waveform Generator in Quantum Experiments

Bachelor's Thesis

Achilleas Sofoklis Lydakis

alydakis@student.ethz.ch

Quantum Optics Group
ETH Zürich

Supervisors:

Jacob Fricke (Quantum Optics Group)

Dr. Kadir Akin

Prof. Tomaso Zambelli

Prof. Tilman Esslinger

December 21, 2025

Acknowledgements

I would like to express my sincere gratitude to Prof. Tilman Esslinger for the opportunity to work in his laboratory and for providing an inspiring and stimulating research environment. I am also deeply grateful to Prof. Tomaso Zambelli, whose supervision, guidance, and continued support were essential throughout the development of this project.

I would further like to thank Jacob Fricke, Bahadır Döğmez, and Dr. Kadir Akin for their invaluable assistance, their willingness to answer countless questions, and the constant support they offered at every stage of this work. I am especially grateful to Dr. Kadir Akin for teaching me Verilog and FPGA programming in one of his courses, which provided essential knowledge that significantly supported my progress in this project.

Finally, I wish to extend my heartfelt gratitude to my family, whose unwavering support has accompanied me throughout my studies. Their patience, encouragement, and belief in my abilities have provided the foundation that made this work possible. Their continuous motivation—whether through thoughtful advice, emotional support, or simply being present—has been an essential source of strength during both the challenging and rewarding moments of this journey.

Abstract

Quantum gas experiments often involve large numbers of experimental devices distributed throughout the laboratory, each requiring precise, low-latency, and highly synchronized digital or analog control signals. These signals are typically transmitted over coaxial cables from a centralized control system, which must reliably coordinate hundreds of timing-critical operations on microsecond or sub-microsecond timescales. To address these demanding requirements, a low-noise eight-channel arbitrary waveform generator (AWG) system was previously developed within our group, consisting of a control PC, an FPGA board, a breakout board, a power-supply and SPI board, and a DAC board equipped with a high-precision 20-bit low-noise DAC chip. In this thesis, I build upon this existing system and extend its FPGA-based control architecture across several key dimensions to better meet the needs of modern quantum gas experiments.

This thesis presents four major advancements that collectively enhance the capability and reliability of the AWG system for quantum gas research.

First, the memory system is improved so that much larger waveforms can be stored and played back, making it possible to run longer experiments and use more complex pulse patterns. Second, a series of speed optimizations are implemented throughout the FPGA firmware and the Ethernet connection, increasing the data-transfer rate to the FPGA, reducing loading times, and improving how quickly new waveforms can be prepared during experiments. Third, a new internal triggering mechanism is implemented to achieve deterministic channel synchronization and configurable control over the number of emitted trigger signals. Finally, we introduce a continuous-playback state machine that enables seamless streaming of waveform data, removing the need to wait between individual waveforms and supporting long, uninterrupted experimental runs.

Together, these improvements provide a faster, more flexible, and more reliable control system for quantum gas experiments. The updated design integrates naturally into the existing laboratory setup and is ready for use in upcoming experimental work.

Contents

Acknowledgements	1
Abstract	2
1 Introduction	5
1.1 Motivation	5
1.2 System Background	5
1.3 Problems in Existing System	6
1.4 Thesis Objectives	7
2 Full Memory Utilization	8
2.1 Previous Memory Limitation	8
2.2 Diagnosing The Problem	9
2.3 Initial Approach to Full Memory Utilization	11
2.4 Implementation	13
2.5 Results	14
3 Speed Improvements	16
3.1 Initial Performance	16
3.2 Identified Bottlenecks	17
3.3 Optimization Approach	19
3.4 Performance Results	24
4 High Precision Internal Triggering	27
4.1 Previous Trigger Problems	27
4.2 Implementation	29
4.3 Results	31
5 Continuous Playback	33
5.1 Limitations of Previous Control	33
5.2 Cyclic Mode DMAs	34
5.3 New State Machine Architecture	36
5.4 Implementation Details	40

CONTENTS

5.5	Results	42
6	Conclusion	47
6.1	Summary of Improvements and Impact on Quantum Experiments	47
6.2	Implementation Notes and Future Considerations	47
6.3	Final Notes	48

Introduction

1.1 Motivation

Quantum gas experiments rely on a central control system that sends many precisely timed digital and analog signals to devices distributed throughout the laboratory. These signals control magnetic fields, laser systems, microwave sources, and other essential components of the experiment. In many setups, commercial arbitrary waveform generators (AWGs) transmit analog signals over long coaxial cables to reach these devices. However, long analog cables can introduce noise, cause signal distortion, and limit the performance and stability of the experiment.

To reduce these issues, the Quantum Optics group developed a custom low-noise eight-channel AWG system that generates analog waveforms locally, close to the experimental hardware. Instead of distributing analog signals across the lab, the system transmits digital waveform data over Ethernet to an FPGA-based control. A high-precision 20-bit digital-to-analog converter (DAC) then converts the data into analog signals directly at the output, improving signal quality and reducing noise pickup.

The development of this system has been a cumulative effort spanning several student projects within the Quantum Optics group. Bahadır Dönmez established the foundational architecture by designing the FPGA-based control framework, custom PCBs, and the concept of distributed, local digital-to-analog conversion [1]. Building on this work, Shijia Chen extended the system to support eight synchronized output channels through redesigned FPGA firmware and breakout boards [2]. Subsequent work by Kevin Mondoloni focused on in-depth characterization and optimization of the DAC board, including detailed noise analysis and component-level tuning [3]. More recently, Marvin Schwander and Julian Caredda investigated system-level noise mechanisms such as ground loops and SPI-induced interference, contributing design recommendations to improve robustness under realistic laboratory conditions [4].

However, as experimental requirements continued to grow in complexity, the existing system proved insufficient for reliable and practical use in the laboratory. This limitation motivated the improvements presented in this thesis, which aim to prepare the system for stable operation in future experiments.

1.2 System Background

The AWG system consists of a control PC, an FPGA, and a set of custom hardware boards that together receive digital waveform data and generate low-noise analog outputs. In this architecture, the FPGA processes the incoming data and drives a high-precision DAC located close to the experimental hardware to minimize noise and signal distortion.

1. INTRODUCTION

The overall operation of the system can be summarized as follows:

- **Control PC and Python Interface:** The user interacts with the system through a Python-based terminal running on the control PC. This interface prepares the metadata and digital waveform samples and sends them to the FPGA via a TCP Server with Ethernet connection. The Python interface also provides basic tools for configuring waveform parameters and initiating playback.
- **FPGA (Zybo Z7-20 Board):** The FPGA acts as the central controller of the AWG system. Upon receiving waveform data from the control PC, it stores the data in internal memory and organizes it for playback. The FPGA also generates the required single-ended SPI signals according to the timing and sequencing rules defined in the firmware. These SPI signals serve as the communication interface between the digital control logic and the DAC hardware.
- **Breakout Board:** The custom-designed breakout board converts the single-ended SPI outputs from the FPGA into low-voltage differential signaling (LVDS) pairs. LVDS transmission helps preserve signal integrity when sending high-speed data to the DAC board. The breakout board also forwards a 24 V DC supply line that powers the downstream electronics.
- **Power-Supply and SPI Conditioning Board:** This intermediate board converts the incoming LVDS signals back into single-ended SPI signals compatible with the DAC. In addition, it regulates the 24 V DC input into multiple voltage rails required by the analog and digital components on the DAC board, ensuring stable power delivery.
- **DAC Board:** At the final stage of the system, the DAC board receives the SPI data and uses a high-precision 20-bit DAC chip to generate the corresponding analog waveform. Because the DAC board is placed close to the target experimental devices, the system avoids long analog transmission lines and therefore reduces noise pickup and distortions.

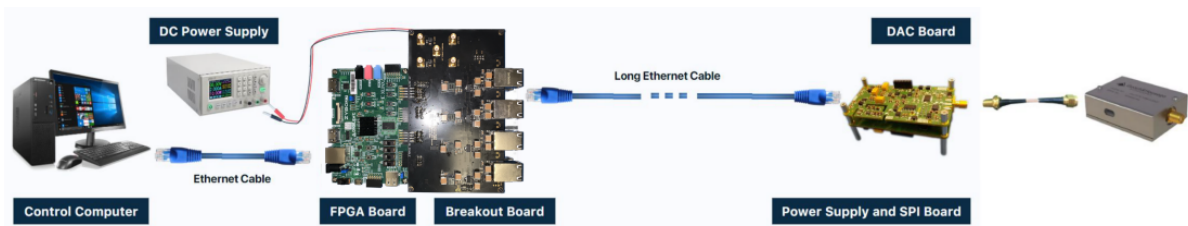


Figure 1.1: Overview of the previous AWG system architecture, taken from Bahadir’s paper [1]

In combination, these components allow the system to generate accurate, low-noise analog outputs with up to eight channels, controlled entirely through programmable digital logic.

1.3 Problems in Existing System

Although the previous AWG system provides a functional FPGA–DAC architecture and low-noise analog output, several technical limitations prevented it from being used reliably in the laboratory. These limitations became more apparent as waveform sizes increased and more complex experimental sequences were considered. The main issues of the existing system are summarized below.

1. INTRODUCTION

Memory Limitation

Although the FPGA board includes 1 GB of DDR memory, the previous system architecture was only able to use a small fraction of it for waveform storage. In practice, most waveforms could occupy less than one quarter of the available DDR memory, and in some cases even waveforms smaller than 65 MB could not be played reliably. This sharply limited the maximum usable waveform size and prevented the system from supporting longer or more complex experimental sequences.

Slow Data Upload

Waveform transmission from the control PC to the FPGA was performed over a simple TCP interface. In practice, the achieved transfer speed was far below the theoretical 1 Gbit/s rate of the Ethernet connection, causing upload times to increase rapidly as waveform size grew. This made it slow to prepare waveforms, especially for large waveforms.

Unstable and Limited Triggering

The original system included only a basic triggering mechanism, which did not guarantee deterministic synchronization between channels under all conditions. Furthermore, the user had limited control over the number and timing pattern of internal trigger pulses. These restrictions made it difficult to coordinate precise timing for multi-channel experiments.

Lack of Continuous Playback

The waveform playback logic relied on a simple state machine that could only handle one waveform at a time. After completing a waveform, the system introduced a pause before the next segment could begin. This prevented seamless transitions between waveforms and made the system unsuitable for experiments requiring continuous, uninterrupted analog output.

1.4 Thesis Objectives

To address the limitations described above, this thesis focuses on four main objectives:

- **Enable waveform files approaching 1 GB** by expanding the usable DDR memory space.
- **Increase data-upload throughput** by redesigning the Ethernet and firmware data path to approach the 1 Gbit/s speed of the Ethernet connection.
- **Implement a deterministic and flexible triggering system** for improved channel synchronization.
- **Introduce continuous waveform playback** by implementing a state machine capable of uninterrupted waveform output.

Together, these improvements transform the previous AWG system into a faster, more flexible, and more robust platform suitable for practical laboratory operation.

Full Memory Utilization

2.1 Previous Memory Limitation

At the time this work began, the memory limitation of the previous AWG system had never been thoroughly analyzed or understood. In practice, it was only known from laboratory experience that larger waveform files would fail to play once they exceeded a certain size. Typically, waveform sizes of around 250–300 MB already caused playback failures, despite the FPGA board being equipped with 1 GB of DDR memory.

Through extensive experimentation, it was eventually determined that the system imposed an implicit limit of 64 MB per channel. Whenever the waveform data for any channel exceeded this size, the generator would crash during playback. Since the AWG supports up to eight channels, this constraint meant that the theoretical maximum amount of data that could be used was $8 \times 64 \text{ MB} = 512 \text{ MB}$, which already corresponds to only half of the available 1 GB DDR memory.

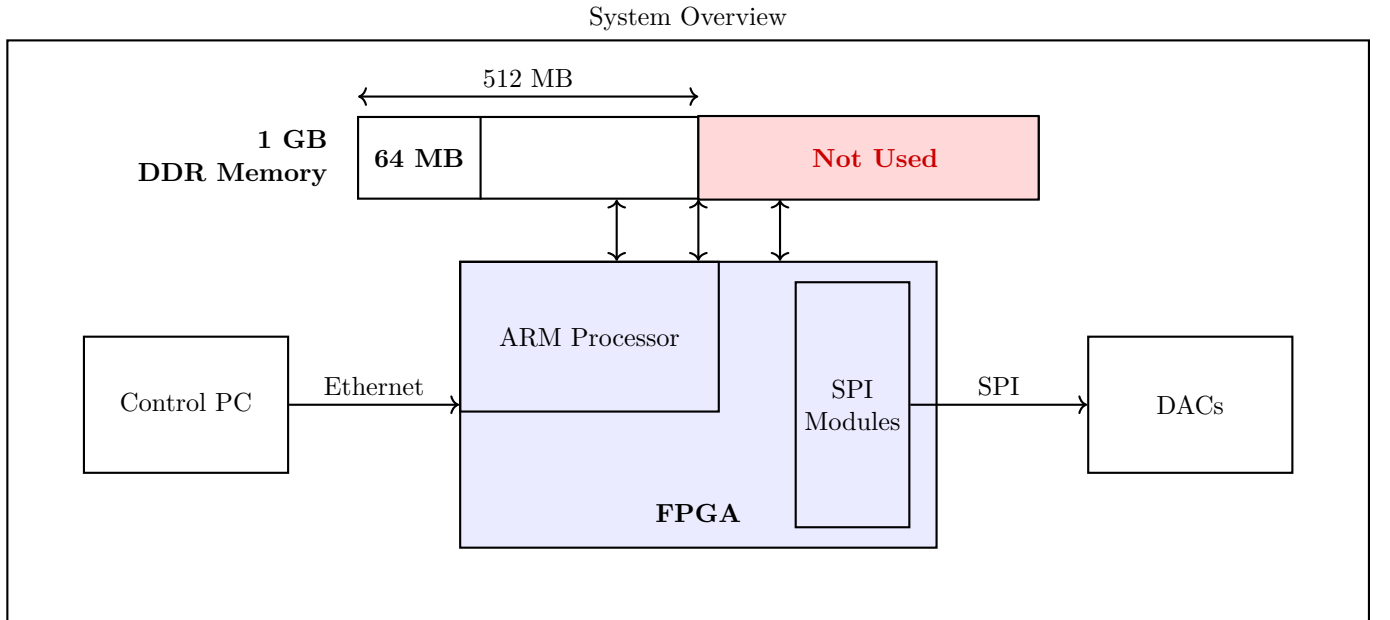


Figure 2.1: High-level system architecture illustrating that only 64 MB can be used per channel and, in the best case, a maximum total of 512 MB is accessible, leaving half of the DDR memory unused.

2. FULL MEMORY UTILIZATION

In practice, this limitation was even more severe. Many experiments do not require all eight channels, and even when multiple channels are used, they rarely contain waveforms of equal length. Because the 64 MB limit applied individually to each channel, typical usage scenarios effectively reduced the usable memory to well below 250–300 MB, consistent with the failures observed in the laboratory. In the worst case—when only a single channel was needed—the system still could not exceed 64 MB for that channel, meaning that as little as 1/16 of the total DDR memory could be used before the device failed completely.

The first task was therefore to understand where the limitation originated and which part of the system restricted the usable memory space. To do this, it was necessary to analyze the entire memory pipeline used during waveform loading and playback.

2.2 Diagnosing The Problem

In the previous system, the control PC sends the waveform samples to the FPGA over an Ethernet connection. The ARM processor on the FPGA receives this data and writes it into the DDR memory, where it is stored until the entire waveform has been transferred. Once all samples are stored in DDR, waveform playback begins. At this stage, the FPGA uses its Direct Memory Access (DMA) engines to transfer data from DDR into a set of FIFOs. A DMA is a hardware module that can move large blocks of data between memory and peripherals without involving the processor for each individual transfer, allowing data to be streamed efficiently at high throughput. The FIFOs then provide a continuous data buffer to the FPGA’s state machine (SPI Modules), which coordinates the timing and formatting required to forward the samples to the DAC board for analog output.

Because this process involves several distinct stages—Ethernet reception, writing data to DDR, reading data from DDR through the DMA engines, filling the FIFOs, and finally transmitting samples to the DAC—any one of these components could, in principle, be responsible for the reduced usable memory range. A careful investigation of each stage was therefore required to determine where the playback failures originated and why only a fraction of the 1 GB DDR memory could be used in practice.

With the problem now clearly identified, the next step was to determine which stage of the memory pipeline caused the limitation. The first task was to verify that the waveform data sent from the control PC actually reached the DDR memory correctly. Using the Vitis Memory Monitor, it was possible to inspect the contents of the DDR and confirm whether the incoming data was being written as expected. These tests showed that the Ethernet transfer and DDR writing stages were functioning correctly: even when sending large waveform files close to one gigabyte in size, all data appeared in memory in the correct order and without corruption.

2. FULL MEMORY UTILIZATION

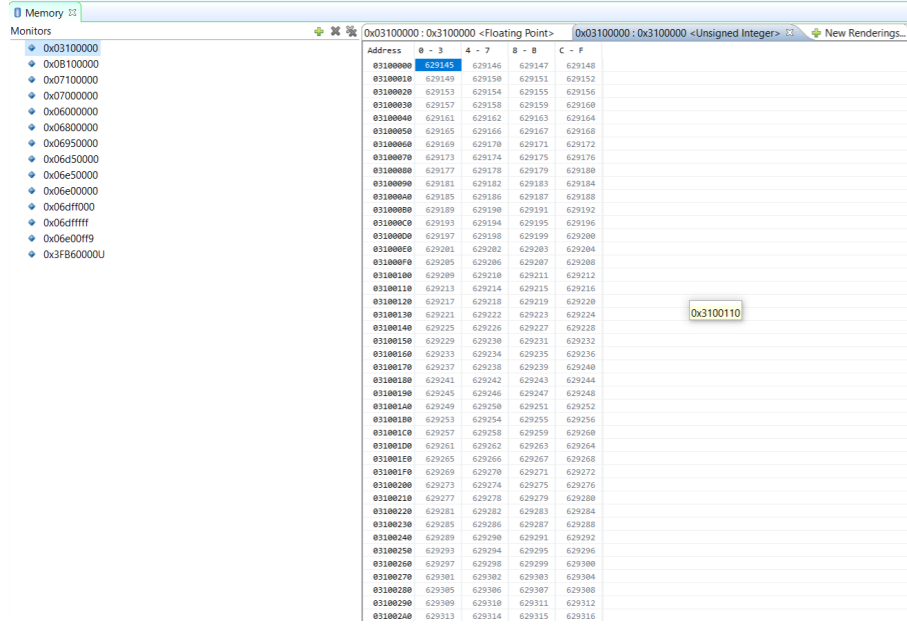


Figure 2.2: Vitis Memory Monitor showing the DDR region starting at 0x03100000. This tool was used to verify that waveform data was correctly written into DDR, confirming that the Ethernet and DDR writing stages were functioning properly.

The addressing scheme used by the ARM processor also supported this interpretation. The processor reserves the address range from 0x0 to 0x3100000 for its own operation, while the remaining space is available for waveform storage. In the previous design, all addresses beyond this point were accessible and could be filled with waveform samples, indicating that DDR capacity itself was not the source of the limitation.

Since the data was clearly being received and stored correctly, the next step was to verify whether the DMA engines were able to read the stored samples from DDR and transfer them into the playback FIFOs. To investigate this, Integrated Logic Analyzer (ILA) cores were added in Vivado to capture the DMA behavior during waveform playback. These measurements revealed the core issue: for larger waveforms, the DMA transactions failed, preventing data from being moved out of DDR and into the FIFOs. This failure explained why playback crashed once per-channel data exceeded the 64 MB threshold, even though the DDR memory contained the correct waveform data. With the DMA failure identified, the next step was to determine why the DMA engines stopped functioning once the waveform size exceeded approximately 64 MB. To investigate this, the DMA configuration of the FPGA design was examined in detail. It quickly became apparent that the DMA controllers were configured with a buffer length register width of only 26 bits.

2. FULL MEMORY UTILIZATION

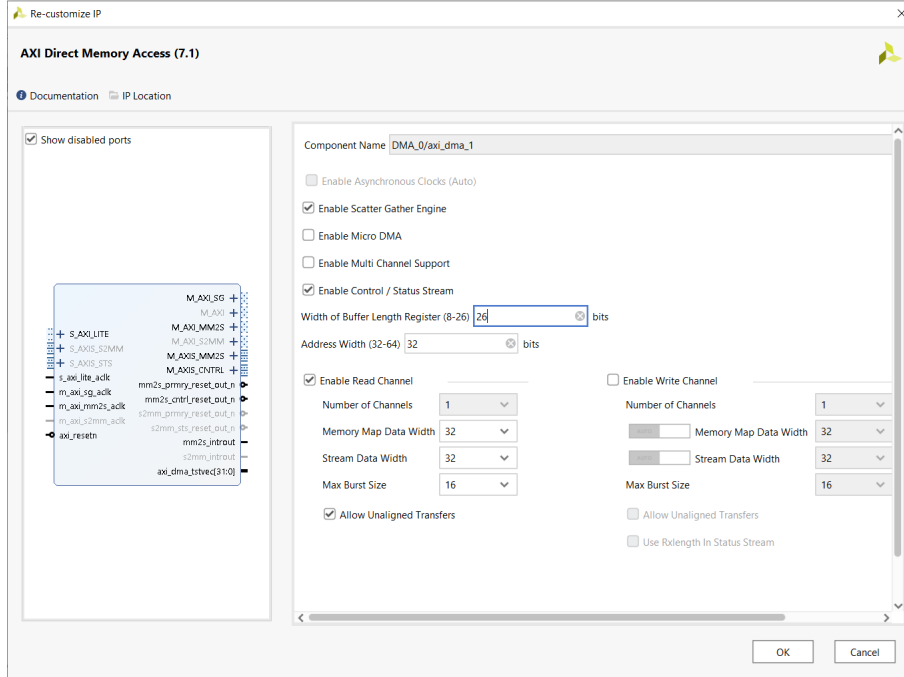


Figure 2.3: Vivado configuration of the AXI DMA. The “Width of Buffer Length Register” is set to 26 bits, which is the maximum value allowed by Vivado (range 8–26 bits).

Since the maximum transferable length is determined by this register, the DMA could address at most:

$$2^{26} = 67,108,864 \text{ bytes} = 64 \text{ MB.}$$

This matched exactly the observed per-channel limitation and confirmed that the DMA configuration—not the DDR memory, the Ethernet input, or the FIFO interface—was responsible for the failure.

Now that the source of the problem had been found, the next step was to redesign the system so that waveform data beyond the 64 MB limit could be transferred reliably via the DMAs from DDR to the playback FIFOs.

2.3 Initial Approach to Full Memory Utilization

The most straightforward fix seemed to be increasing the DMA buffer-length register from 26 bits to 30 bits, allowing the controller to address the full 1 GB waveform space. However, the AXI DMA cores on our board did not support wider length fields. The design was already using the maximum supported width, so extending the register was not an option. This immediately ruled out any solution based on simply enlarging the DMA’s addressable transfer range.

With the hardware limit fixed at 26 bits, the system needed a method to move waveform data in segments small enough for the DMA to handle. The only viable path forward was therefore to implement a chunking mechanism: instead of transferring an entire waveform in a single DMA operation, the FPGA would receive it in 64 MB blocks, each individually compliant with the DMA’s maximum transaction size.

The original software flow relied on a single AXI DMA simple transfer per channel. This design assumed that the waveform occupied a contiguous buffer whose length could be passed

2. FULL MEMORY UTILIZATION

directly to the DMA engine. Once waveforms grew beyond 64 MB, the length argument exceeded the DMA’s 26-bit limit. As a result, the transfer call attempted to configure the DMA with an invalid size, causing exceptions and immediate playback failure.

The first idea was therefore to keep the existing simple-transfer workflow but call it repeatedly, each time with a chunk smaller than the 64 MB threshold. Instead of issuing one large DMA transaction, the ARM processor iteratively performed many simple transfers, walking through the waveform buffer in memory and dispatching each block sequentially. This approach would in theory preserve the overall architecture of the previous system while allowing large waveforms to be streamed.

However, this approach quickly proved unworkable once it was tested in the full playback pipeline. The problem came from how the ARM processor and the DMA engines interact during a simple transfer. When the ARM issues a simple-transfer command, the DMA begins pushing data toward the playback FIFOs, but the FIFO contents are not consumed until the SPI modules receive their own start signal. Until that moment, the FIFO remains full, and the DMA has no way to complete the transfer.

Because of this, the first DMA transaction never actually reaches its “transfer complete” state. If the software immediately issues another simple transfer—even if it is intended for the next 64 MB chunk—the new call overwrites the internal state of the DMA before the previous transfer has progressed. In practice, the DMA repeatedly restarts, and only the final chunk ever gets delivered to the playback path.

Waiting for the DMA to finish is also not an option. The DMA cannot complete until the SPI modules begin consuming data, but the SPI modules cannot begin until the dma transfers have been started. This circular dependency means that the ARM can never wait for one chunk to finish before launching the next.

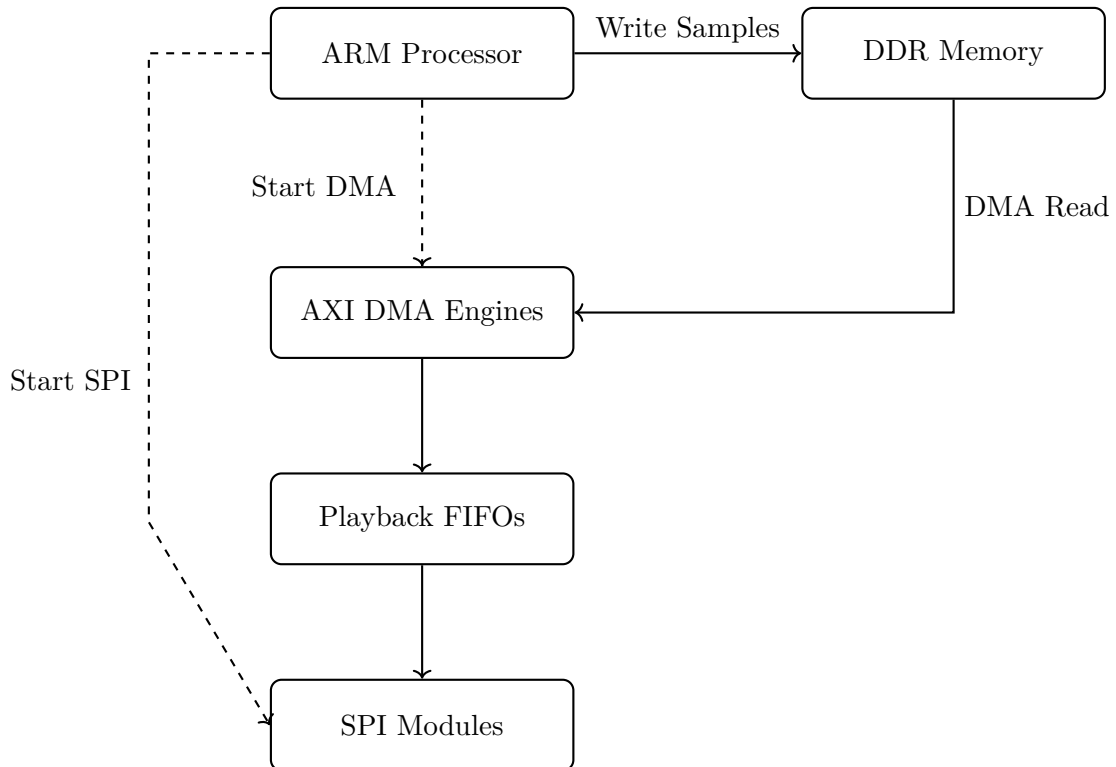


Figure 2.4: Simplified data and control pipeline of the original AWG system.

2. FULL MEMORY UTILIZATION

For these reasons, the strategy of using multiple simple DMA transfers in sequence could not reliably stream large waveforms. Even though it respected the 64 MB transfer limit, the control flow of the system made it impossible to chain simple transfers without corrupting or overwriting data. A different approach was needed to move beyond the 64 MB barrier.

2.4 Implementation

With the 64 MB DMA transfer-length limit identified and simple-mode chaining proven infeasible, the system was redesigned around AXI DMA Scatter-Gather (SG) mode.

SG mode allows the DMA engine to process a sequence of smaller transfer segments instead of relying on a single large transaction. Each segment is described by a descriptor stored in memory, containing the DDR start address, the segment length, and a pointer to the next descriptor. By following these pointers, the DMA hardware can autonomously step through a chain of descriptors and stream data in multiple valid-sized chunks without further interaction from the processor.

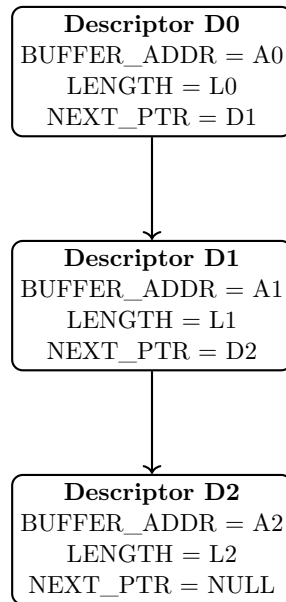


Figure 2.5: Scatter-Gather DMA descriptor structure used to overcome the 64 MB transfer limit. This ring allows the DMA engine to autonomously fetch and process multiple waveform segments in sequence, enabling continuous streaming of arbitrarily large waveforms beyond the single-transfer size limit.

In this design, the ARM processor constructs the full descriptor chain before playback begins. The waveform buffer is divided into blocks smaller than the 64 MB limit, and for each block a descriptor is created and placed into DDR memory. All descriptors are arranged consecutively so that the DMA engine can fetch one, transfer its associated data into the playback FIFOs, update status information, and immediately move to the next. The DMA automatically pauses when the FIFOs are full and resumes once space becomes available, ensuring continuous streaming even though the waveform is internally processed in separate segments.

This approach solves the original limitation because the DMA no longer needs to handle a transfer larger than 64 MB at once. Instead, it performs many smaller transfers, each within

2. FULL MEMORY UTILIZATION

the hardware limit, while still producing a seamless output stream. Most importantly, once playback is started, the entire transfer proceeds without any ARM processor involvement: the DMA engine autonomously traverses the descriptor chain and maintains the data flow until all waveform segments have been streamed.

This behavior highlights the core advantage of the SG-based design: once the descriptor ring has been submitted and the DMA is started, the hardware is fully capable of streaming the entire waveform buffer on its own. The DMA engine fetches each descriptor in order, reads the corresponding block from DDR, and advances to the next descriptor automatically, requiring no supervision or additional configuration from the ARM processor. After initiating the DMA transfer, the ARM's role is limited to enabling the playback FIFOs and starting the SPI modules that feed the DAC. From that point onward, waveform playback is sustained entirely by the DMA hardware, which continues cycling through all descriptors until the full dataset has been delivered. This separation of responsibilities removes the processor from the timing-critical data path and is what ultimately allows the system to handle waveform sizes far beyond the original 64 MB limit.

2.5 Results

The measurements demonstrate that the Scatter–Gather implementation completely removes the 64 MB per-channel limitation of the previous system. Whereas the old simple-transfer DMA design consistently failed whenever any individual channel exceeded 64 MB, the new SG-based pipeline successfully streamed all tested waveform configurations, including cases where a single channel surpassed this limit as well as cases where the combined waveform size approached the full DDR capacity.

In the old system, exceeding the 64 MB limit on a single channel caused playback failure even if the total waveform size remained modest. With the new implementation, the specific distribution of samples across channels no longer matters; only the total number of samples must remain below the safe operational limit of samples. This significantly simplifies the control-PC interface while allowing channels to contain arbitrarily large and uneven waveform lengths without introducing individual memory constraints.

From a memory-capacity perspective, the SG implementation enables almost the entire DDR memory to be used for waveform storage. The ARM processor reserves memory up to address 0x03100000, which corresponds to

$$0x03100000 = 51,380,224 \text{ bytes.}$$

Additionally, each of the eight DMA channels requires approximately 600 kB of descriptor meta-data:

$$8 \times 600,000 \text{ bytes} = 4,800,000 \text{ bytes.}$$

Subtracting both reserved regions from the available DDR capacity of 1 073 741 824 bytes (1 GB) yields

$$1,073,741,824 - 51,380,224 - 4,800,000 = 1,017,561,600 \text{ bytes,}$$

which is the space available for waveform samples. Since each sample occupies 4 bytes, the theoretical maximum number of addressable samples is

$$\frac{1,017,561,600}{4} = 254,390,400 \text{ samples.}$$

2. FULL MEMORY UTILIZATION

To ensure safe operation under all usage conditions, the control PC enforces a conservative upper limit of 250 million samples. As long as the total number of samples remains below this boundary, any number of channels and any distribution of waveform lengths is supported, and the DMA engine streams all data reliably and without error.

Test Case	Old System	New SG System
All channels small (e.g. 8×10 MB = 80 MB)	Success	Success
All channels < 64 MB (total approx. 300 MB)	Success	Success
One channel > 64 MB (e.g. 1×100 MB)	Fail	Success
>512 MB total waveform	Fail	Success
approx. 1 GB total waveform	Fail	Success

Table 2.1: Playback behavior of the old simple-transfer DMA system versus the new Scatter–Gather implementation across representative waveform configurations.

With the conservative limit of 250 million samples, the system uses

$$250,000,000 \times 4 \text{ bytes} = 1,000,000,000 \text{ bytes}$$

of waveform data. Compared to the total DDR capacity of

$$1,073,741,824 \text{ bytes},$$

this corresponds to

$$\frac{1,000,000,000}{1,073,741,824} \approx 93.1\%.$$

Thus, even with the imposed safety limit, the system is capable of using more than 93% of the full DDR memory space for waveform storage while still guaranteeing that boundary overflows cannot occur.

These results confirm that Scatter–Gather mode enables stable, continuous playback of large waveform datasets, fully eliminates the per-channel memory constraints of the previous architecture, and allows nearly the entire DDR address space to be used for waveform storage. Once the descriptor ring is initialized and the DMA is started, playback proceeds autonomously without further ARM processor intervention, demonstrating the robustness and scalability of the redesigned memory pipeline.

Speed Improvements

3.1 Initial Performance

After the memory issues were resolved, the next objective was to address the data-transfer speed. The Ethernet interface, based on a 1 Gbit/s link, should theoretically sustain up to 125 MB/s. With a maximum of 250 million samples, each occupying 4 bytes, the total maximum payload is:

$$250 \times 10^6 \times 4 = 1 \times 10^9 \text{ bytes} \approx 1 \text{ GB.}$$

At 125 MB/s, transmitting 1 GB of data should require roughly:

$$\frac{1 \text{ GB}}{125 \text{ MB/s}} \approx 8 \text{ seconds.}$$

However, initial measurements showed that the actual throughput was more than an order of magnitude lower than this theoretical expectation. A plot of these baseline measurements is presented below:

Samples	Sample Calculation (s)	Total Time (s)
1 million	0.7	2.5
10 million	1.4	9
20 million	2.9	15.5
40 million	7.3	28.5
80 million	13.3	55
100 million	17	69
130 million	21	89
150 million	25	108
200 million	33.4	137
250 million	40	178

Table 3.1: Initial throughput measurements for different sample counts, showing sample calculation time from the control PC and total transfer time .

The column labeled *Sample Calculation* represents the time the control PC required to numerically generate all sample values in Python. For these tests, the waveform consisted of a sine wave and a sawtooth wave (a linearly rising signal that resets to zero once it reaches its

3. SPEED IMPROVEMENTS

maximum amplitude). The measured values therefore reflect only the computational overhead of producing the samples before transmission.

The *Total Time* column corresponds to the full end-to-end latency: from the moment the control PC begins calculating the samples until the first sample is actually played on the DAC board. This includes calculation time, data transfer, buffering, and any processing delays in the system. Thus, the total time captures the effective perceived latency of the entire signal-generation path.

To our knowledge, the only components expected to contribute noticeable delay to our system were the Ethernet transfer and the sample generation on the control PC. All other data paths in the system should have been fast enough to add negligible overhead. With the Ethernet link alone requiring about 8 seconds to transmit the maximum 250 million samples, this formed our theoretical performance limit.

However, the measured performance was far slower: for the maximum 250 million samples, the complete process—including waveform generation—took around 178 seconds instead of the expected 8 seconds, more than a twentyfold slowdown. This discrepancy made it necessary to verify whether the Ethernet link was truly operating at 1 Gbit/s and to determine whether additional bottlenecks were present in the data path.

3.2 Identified Bottlenecks

To investigate the unexpectedly long transfer time, a series of experiments was conducted to measure each stage of the data path separately. All measurements were performed using the worst-case scenario of 250 million samples, since the measured processing times scaled approximately linearly with the number of samples. This meant that by minimizing the worst-case performance, all smaller waveforms would benefit proportionally. By isolating the individual contributions of waveform generation, memory writing, and Ethernet transfer, the main bottlenecks in the system could be identified.

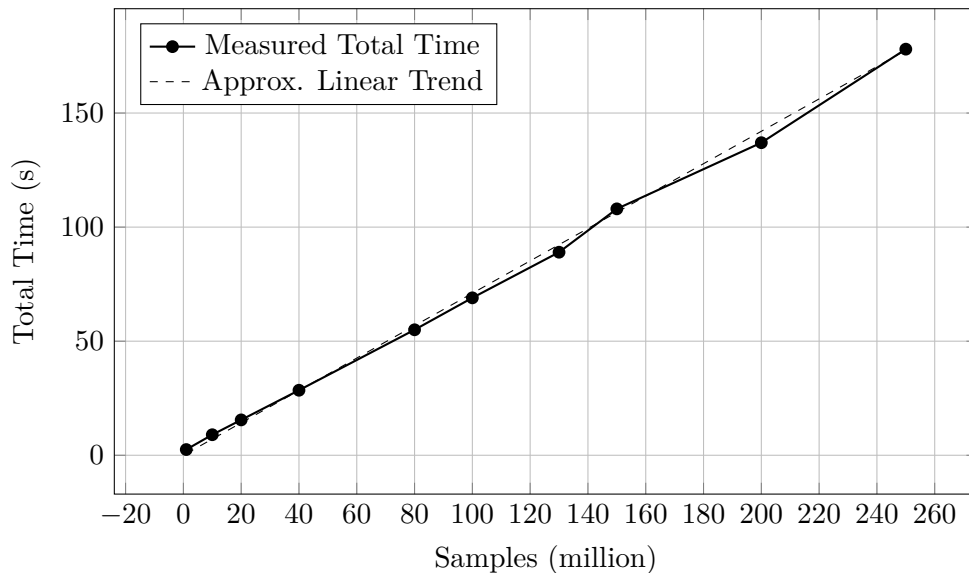


Figure 3.1: Measured end-to-end latency as a function of sample count, showing an approximately linear relationship.

3. SPEED IMPROVEMENTS

The first bottleneck we identified was the waveform generation on the control PC. Python’s sample-by-sample computation introduced a significant delay, and producing the full 250 million samples required up to 40 seconds of processing time. This duration was measured directly in Python using the `time` module, which recorded the interval between the start and end of the waveform generation routine. The measurements showed that this computation alone already contributed a substantial portion of the overall latency.

The second bottleneck was the write path on the ARM processor. Once the TCP server inside the FPGA received the incoming Ethernet data, the ARM processor had to copy the samples into DDR memory, and this step turned out to be significantly slower than expected. The duration of this memory-copy operation was measured using the `XTime` hardware cycle counter, which captured the exact number of cycles spent writing the received data into DDR. After converting the cycle count to seconds, we found that this stage alone required approximately 28 seconds. This showed that TCP handling, user–kernel transitions, and unoptimized memory writes introduced considerable overhead, further extending the total latency.

Finally, the Ethernet connection itself proved to be a bottleneck. Even after optimizing the waveform generation and the memory-write path on the ARM processor, the transfer speed remained far below the theoretical 1 Gbit/s limit. The Ethernet overhead was determined indirectly by subtracting the measured Python computation time and the ARM DDR write time from the total observed latency of 178 seconds for 250 million samples. With approximately 40 seconds spent on waveform generation and 28 seconds on DDR writing, the remaining ~ 110 seconds corresponded to the effective Ethernet transfer time and its associated software-handling delays, making it the dominant limiting factor once the other issues were addressed.

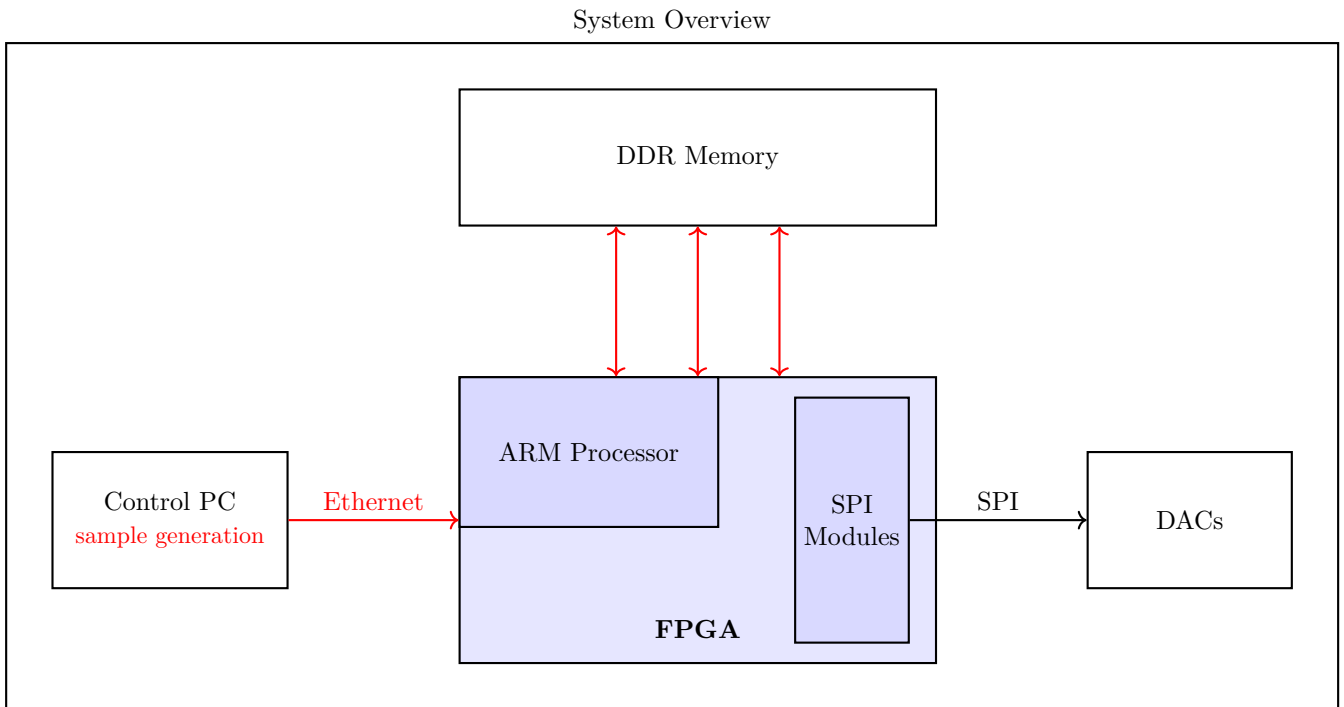


Figure 3.2: High-level system architecture. The highlighted bottlenecks in the system are: (1) sample generation inside the Control PC, (2) the data transfer to and from the DDR memory, and (3) the Ethernet link speed between the Control PC and the FPGA.

3. SPEED IMPROVEMENTS

3.3 Optimization Approach

To address the performance issues identified in the previous section, each bottleneck had to be resolved individually. The optimization process was carried out step by step, targeting waveform generation, memory writing, and Ethernet transfer independently so that the impact of each improvement could be clearly measured and verified.

Python Optimization

The first optimization focused on the waveform generation in Python. This was both the easiest bottleneck to address and, unfortunately, the least rewarding in terms of overall improvement. By replacing the sample-by-sample computation with NumPy’s vectorized operations—specifically using `np.arange` for the sawtooth wave calculation and `np.sin` for the sine wave calculation to generate the waveform—the generation time was reduced from 40 seconds to approximately 18 seconds. Although this halved the computation time, the improvement remained relatively small in the context of our initial twentyfold slowdown; 18 seconds is still a considerable portion of the total latency. We also tested the generation routine on a faster machine, but saw no meaningful reduction beyond this point, suggesting that we may be limited by memory bandwidth rather than raw CPU performance.

A C/C++ implementation could be considerably faster, potentially allowing much quicker sample generation. The Quantum Optics lab’s C-based server likely benefits from this, but such an approach was outside the scope of our setup, and with the existing Python–NumPy environment, no further practical optimizations appear feasible.

DDR Memory Optimization

The second part of the optimization concerned the DDR memory writing on the ARM processor. In our setup, the control PC communicates with the ARM through a TCP Ethernet connection, sending a continuous stream of packets of roughly 1500 bytes each (plus protocol overhead). Each time the ARM receives one of these packets, it first stores the payload in a temporary buffer and then writes the data into DDR memory. For large waveforms this operation must be repeated a very large number of times; for example, a 1 GB waveform corresponds to $\frac{1\text{GB}}{1500\text{ bytes}} \approx 700\,000$ packets, meaning the ARM must perform hundreds of thousands of DDR write operations during a single transfer. It is therefore crucial that these continuous writes into DDR memory are performed as quickly as possible so that incoming packets can be processed without delay and the data pipeline remains saturated.

However, it was observed that the previous implementation used the `Xil_Out32` command for writing each chunk of data into memory. This function introduces significant overhead, causing unnecessary delays and slowing down the entire DDR write path. After reviewing the code, we replaced this call with a far more efficient `mempcpy` operation, which minimizes the time spent writing incoming data to DDR and allows the ARM processor to keep up with the high rate of incoming packets. A code excerpt of the previous implementation will be shown below.

3. SPEED IMPROVEMENTS

```
/* Copy the remaining samples in the payload into DDR. */
while(g_idx_payload < len_payload/4) {
    Xil_Out32(MEM_BASEADDR + 4 * g_memory_count, payload[g_idx_payload]);
    /* Increment memory counter and payload index by 1 and decrease remaining bytes by 4. */
    g_idx_payload += 1;
    g_memory_count += 1;
    *rem_bytes_channel -= 4;
}
```

Figure 3.3: Previous DDR write implementation using `Xil_Out32`, which writes data word-by-word and introduces significant overhead.

```
1371 /* ---- Fast single memcpy ---- */
1372 | memcpy((void *)write_addr, (void *)&payload[g_idx_payload], bytes_to_copy);
1373 | /* ---- */
1374
1375 /* Update tracking counters */
1376 g_idx_payload += bytes_to_copy / 4;
1377 g_memory_count += bytes_to_copy / 4;
1378 *rem_bytes_channel -= bytes_to_copy;
1379
```

Figure 3.4: Optimized DDR write implementation using `memcpy`, which performs fast block-copy operations and reduces write latency.

The reason `memcpy` is significantly faster is that it performs memory transfers using highly optimized, architecture-specific routines. Instead of writing data word-by-word in a loop—as `Xil_Out32` effectively does—`memcpy` moves large contiguous blocks of memory at once, often using wide bus transfers, burst operations, and CPU instructions that copy multiple bytes per cycle. This reduces function-call overhead, minimizes instruction count, and makes far better use of the available memory bandwidth.

This optimization had a substantial impact on performance. Replacing the `Xil_Out32` loop with the `memcpy` implementation reduced the DDR write time for the maximum waveform size of 250 million samples from approximately 28 seconds to about 8 seconds. This corresponds to a speedup of roughly $\frac{28}{8} \approx 3.5\times$, allowing the ARM processor to keep pace with incoming packets far more effectively and significantly reducing the memory-handling contribution to the overall latency.

Ethernet Optimization

The final and most critical issue to address was the Ethernet transfer speed. Before any optimizations, the effective Ethernet overhead for the worst-case waveform of 250 million samples was approximately 110 seconds, even though a 1 Gbit/s link should theoretically require only about 8 seconds to transmit the same amount of data. This corresponds to a slowdown of more than fourteen-fold compared to the expected performance, making the Ethernet path by far the dominant contributor to the total latency. Consequently, improving this part of the system became essential, as any meaningful reduction in overall transfer time depended primarily on accelerating the Ethernet communication.

To improve the Ethernet performance, it was necessary to modify the BSP (Board Support Package) settings within the platform’s `.spr` file in Vitis. The BSP defines the low-level software configuration for the ARM processor and the LWIP stack used by the TCP server, and changing these settings directly affects how the system handles incoming Ethernet traffic. In our case, the parameters that ultimately had the largest impact on throughput were found in the LWIP configuration: the `tcp_wnd` value under the TCP options and the `PBUF_POOL_SIZE` under the

3. SPEED IMPROVEMENTS

pbuf options. Adjusting these settings was necessary for maximizing the potential of the Ethernet connection.

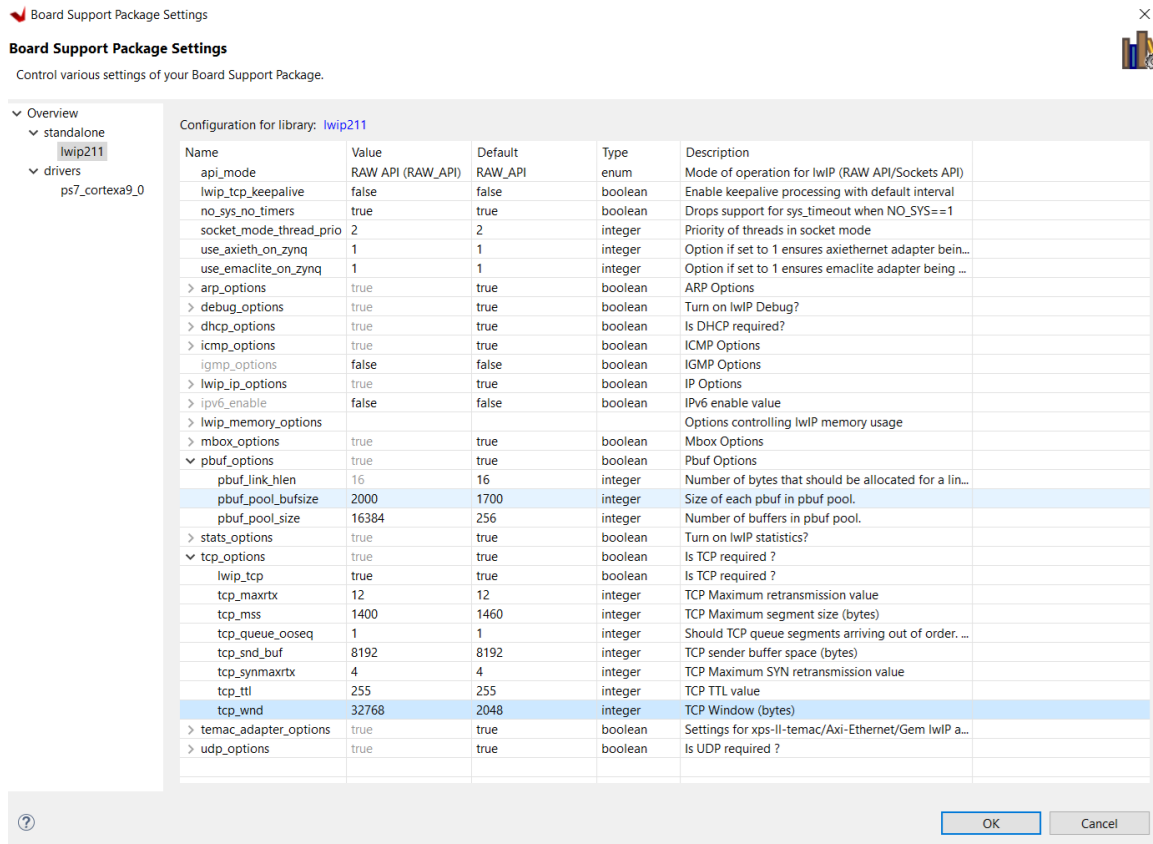


Figure 3.5: BSP settings in the Vitis platform .spr file, showing the LWIP configuration parameters that were modified to improve Ethernet performance.

The first BSP parameter that we examined was the `PBUF_POOL_SIZE`. This setting controls how many packet buffers LWIP can allocate at once. Increasing it effectively enlarges the available buffering capacity, allowing more incoming data to be stored before processing and reducing the number of times the system must stall while waiting for free buffers. In principle, this should provide a small improvement in throughput. However, in practice the change from the default value of 1700 to 2000 resulted in only a very minor speedup—so small that it was negligible compared to the improvements achieved through the second parameter modification.

In TCP communication, every transmitted segment must eventually be acknowledged by the receiver to confirm successful delivery. The sender is only allowed to have a limited amount of un-acknowledged data “in flight” at any moment. Once this limit is reached, the sender must pause transmission until new acknowledgments arrive. The size of the TCP receive window defines this limit: it specifies how many bytes the receiver is prepared to accept before an acknowledgment becomes mandatory. If the window is small, acknowledgments must be exchanged frequently, causing the sender to stop and wait after relatively small bursts of data. These stalls can severely limit throughput, especially in high-bandwidth links where the sender could otherwise transmit data much faster than the acknowledgments are returned.

To understand why the TCP window size has such a strong influence on throughput, it is

3. SPEED IMPROVEMENTS

useful to recall how acknowledgements regulate the flow of data in TCP. The sender is only allowed to transmit an amount of unacknowledged data limited by the TCP receive window (`tcp_wnd`). Once this window is filled, the sender must stop and wait for an acknowledgement (ACK) before sending the next portion of data. As a consequence, the maximum achievable throughput is bounded by the classical window–bandwidth relationship

$$\text{Throughput}_{\max} = \frac{\text{TCP window size}}{\text{RTT}},$$

where RTT denotes the round-trip time between sender and receiver. This formula expresses that the sender can transmit at most one full window of data per RTT. For a total payload of size S , the corresponding total transfer time in the ACK-limited regime becomes

$$t_{\text{total}} = \frac{S}{\text{Throughput}_{\max}} = \frac{S \cdot \text{RTT}}{\text{tcp_wnd}}.$$

If the receive window is small, then `tcp_wnd` appears in the denominator, and the required transfer time increases proportionally. In this case the sender repeatedly transmits a small burst of data, waits for an ACK, and only then continues. These stalls severely limit throughput, especially on a fast link such as Gigabit Ethernet, where the sender would otherwise be able to transmit continuously.

As an illustrative example, the worst-case waveform of approximately 1 GB (250 million samples) required about 110 seconds to transfer over the Ethernet link. The corresponding measured throughput is therefore

$$\text{Throughput}_{\text{meas}} = \frac{1 \times 10^9 \text{ bytes}}{110 \text{ s}} \approx 9.09 \times 10^6 \text{ bytes/s} \approx 9 \text{ MB/s}.$$

Using the TCP throughput relation

$$\text{Throughput} = \frac{\text{tcp_wnd}}{\text{RTT}},$$

and the initial window size of `tcp_wnd` = 2048 bytes, the effective RTT implied by our measurements becomes

$$\text{RTT} = \frac{2048}{9.09 \times 10^6} \approx 2.25 \times 10^{-4} \text{ s} = 0.225 \text{ ms}.$$

With such a small window, TCP can send only 2048 bytes per RTT before waiting for an acknowledgement, severely limiting the transmission rate. This explains why the measured throughput was far below the theoretical 1 Gbit/s limit.

If we instead consider a window size of 32 kB (the value used after optimization), the maximum achievable throughput for the same RTT becomes

$$\text{Throughput}_{32\text{k}} = \frac{32768}{0.000225} \approx 1.45 \times 10^8 \text{ bytes/s} \approx 145 \text{ MB/s}.$$

Since this exceeds the physical limit of Gigabit Ethernet (125 MB/s), TCP flow-control is no longer the bottleneck, and the minimum achievable transfer time is determined solely by the link rate:

$$t_{\text{ideal}} = \frac{1 \text{ GB}}{125 \text{ MB/s}} = 8 \text{ s}.$$

These calculations explain both the very poor initial performance and the significant improvement obtained after increasing the TCP receive window in the BSP configuration.

3. SPEED IMPROVEMENTS

With these results, the impact of enlarging `tcp_wnd` becomes evident. A larger TCP receive window allows significantly more data to remain in flight before an acknowledgement is required, reducing the number of stalls caused by ACK waiting and preventing the link from operating in short, interrupted bursts. By keeping the pipeline filled, the sender can sustain continuous transmission, leading to a substantial increase in effective throughput and enabling the Ethernet performance to approach the theoretical limit of the 1 Gbit/s link.

A final contributing bottleneck related to Ethernet performance was of a physical nature. During the initial experiments, the control PC was connected to the FPGA board through a USB-to-Ethernet adapter rather than a direct Ethernet-to-Ethernet link. For the maximum waveform size of 250 million samples, this adapter introduced a measurable throughput limitation, accounting for roughly 8 seconds of additional overhead in the end-to-end transfer time. After switching to a direct Ethernet connection, this overhead was removed, demonstrating that the adapter itself had been imposing a non-negligible restriction on the achievable data rate.



Figure 3.6: USB-to-Ethernet adapter (Ranz Hi-Speed USB to Fast Ethernet Adapter), similar to the device used during the initial laboratory experiments. The use of this adapter introduced additional throughput overhead during the transfer of 250 million samples.

To further isolate the behaviour of the TCP server, an additional experiment was performed in which the DDR writing phase was completely bypassed. In this test, the server simply received all incoming packets without storing them to memory. After subtracting the 18 seconds required for sample generation on the control PC, the remaining transfer time was approximately 8.5 seconds for the full 250 million-sample waveform. This measurement confirmed that, once the BSP settings were corrected and the physical bottleneck removed, the TCP server was operating close to its practical throughput limit, and no significant additional speed improvements could be expected from the Ethernet path alone.

Using the measured transfer time of 8.5 seconds for the full 250 million-sample waveform, the effective Ethernet throughput can be calculated directly. With a total payload of 1×10^9 bytes, the achieved transfer rate is

$$\text{Throughput} = \frac{1 \times 10^9 \text{ bytes}}{8.5 \text{ s}} \approx 1.176 \times 10^8 \text{ bytes/s} \approx 117.6 \text{ MB/s} \approx 0.94 \text{ Gb/s}.$$

Since the theoretical maximum throughput of a 1 Gbit/s Ethernet link is 125 MB/s, the achieved speed corresponds to

$$\frac{117.6}{125} \times 100\% \approx 94.1\%.$$

3. SPEED IMPROVEMENTS

This result shows that, after the BSP adjustments and the removal of the physical adapter bottleneck, the TCP server now operates at approximately 94% of the theoretical limit of the Gigabit Ethernet link, confirming that the Ethernet path is performing close to its maximum practical capacity; compared to the initial throughput of approximately 9 MB/s, the achieved throughput of 117.6 MB/s represents an improvement by a factor of about 13.

3.4 Performance Results

Finally, after applying all of the improvements described in the previous sections, it is time to evaluate the resulting performance of the system. The following measurements summarize the behaviour of the optimized design and allow a direct comparison with the initial, unoptimized implementation.

Samples	Sample Calculation (s)	Total Time (s)
1 million	0.69	1.13
10 million	1.3	3.3
20 million	2.0	3.7
40 million	3.35	6.0
80 million	6.1	11.3
100 million	7.5	14.0
130 million	9.7	17.7
150 million	11.8	21.5
200 million	14.8	26.87
250 million	18.0	32.0

Table 3.2: Final optimized system performance for different sample counts.

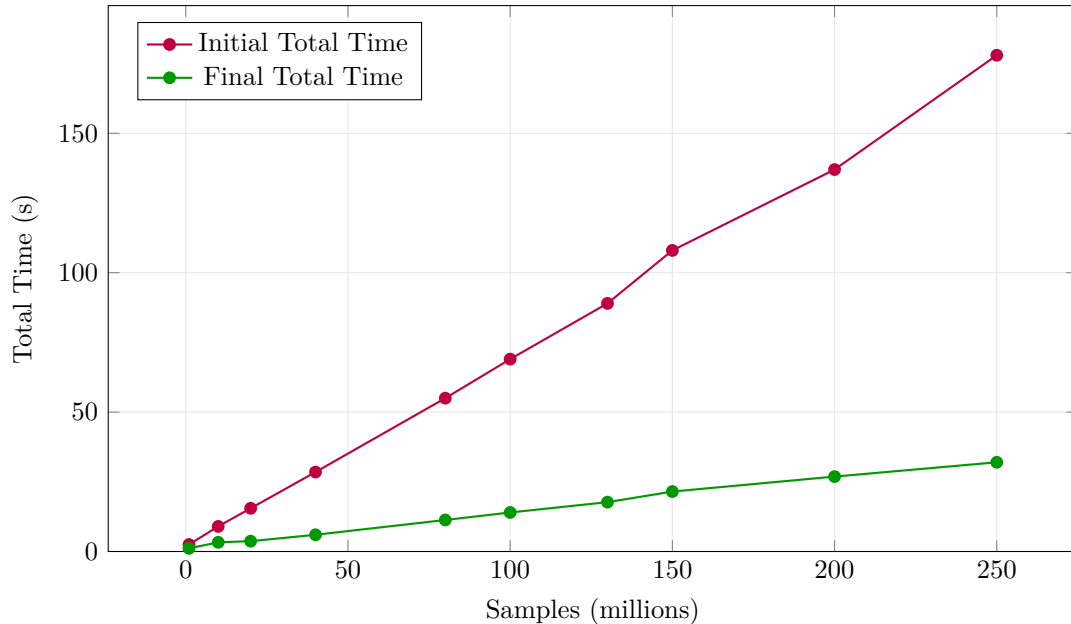


Figure 3.7: Comparison of initial and final total transfer time for all waveform sizes.

3. SPEED IMPROVEMENTS

As the results clearly show, the applied optimizations improved the overall transfer speed by a very large margin. Across all waveform sizes, the end-to-end time was reduced by approximately a factor of six compared to the original implementation.

It is also worth noting that for the maximum waveform of 250 million samples the sum of the individually measured delays—18 seconds for sample generation, 8 seconds for DDR writing, and 8.5 seconds for Ethernet transfer—amounts to 34.5 seconds, which is slightly higher than the final measured end-to-end time of 32 seconds. This difference arises because some operations occur in parallel: while the ARM processor is writing data to DDR memory, the TCP server continues receiving subsequent packets. As a result, part of the DDR-write latency overlaps with the Ethernet-transfer latency, effectively reducing the visible end-to-end delay.

Finally, a further relevant observation is how the relative contribution of the sample-generation (Python) phase changed between the initial and final implementations. In the initial system, the sample calculation accounted for only about $\approx 22\%$ of the total end-to-end time. After all optimizations were applied, this changed significantly: in the final implementation, the sample-generation step represents approximately $\approx 56\%$ of the total latency. This shift reflects the fact that the other major bottlenecks were largely eliminated, leaving the waveform calculation on the control PC as the predominant remaining limitation.

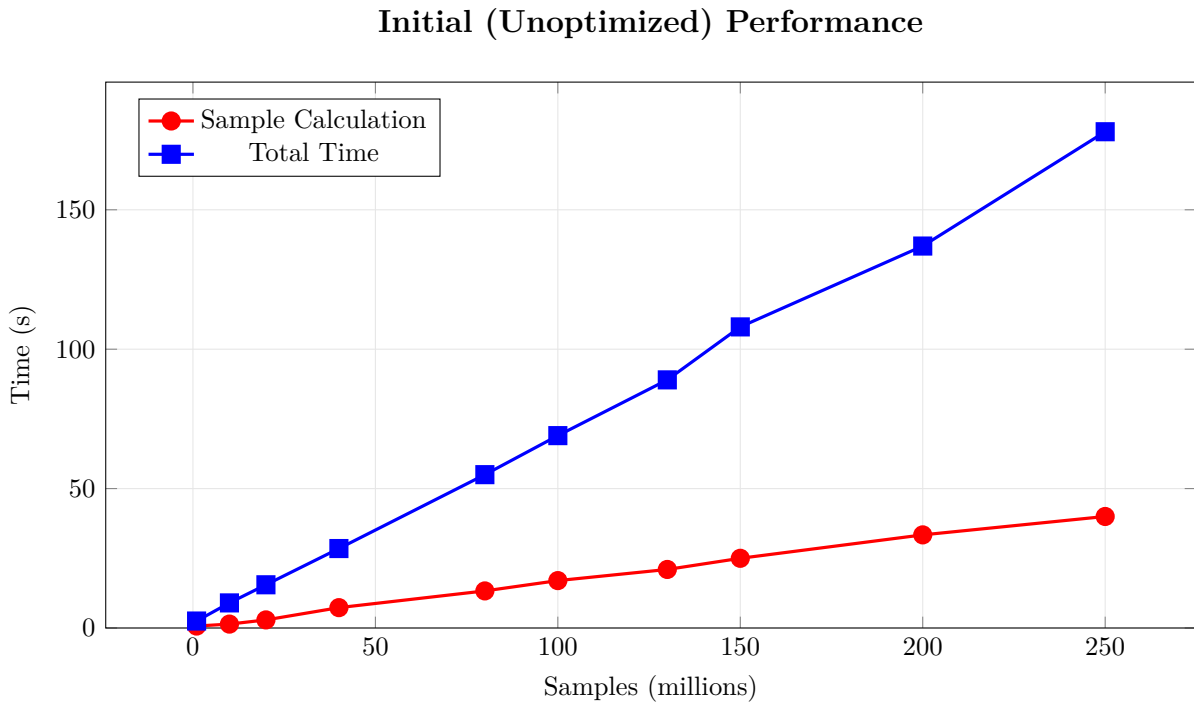


Figure 3.8: Initial (unoptimized) performance.

3. SPEED IMPROVEMENTS

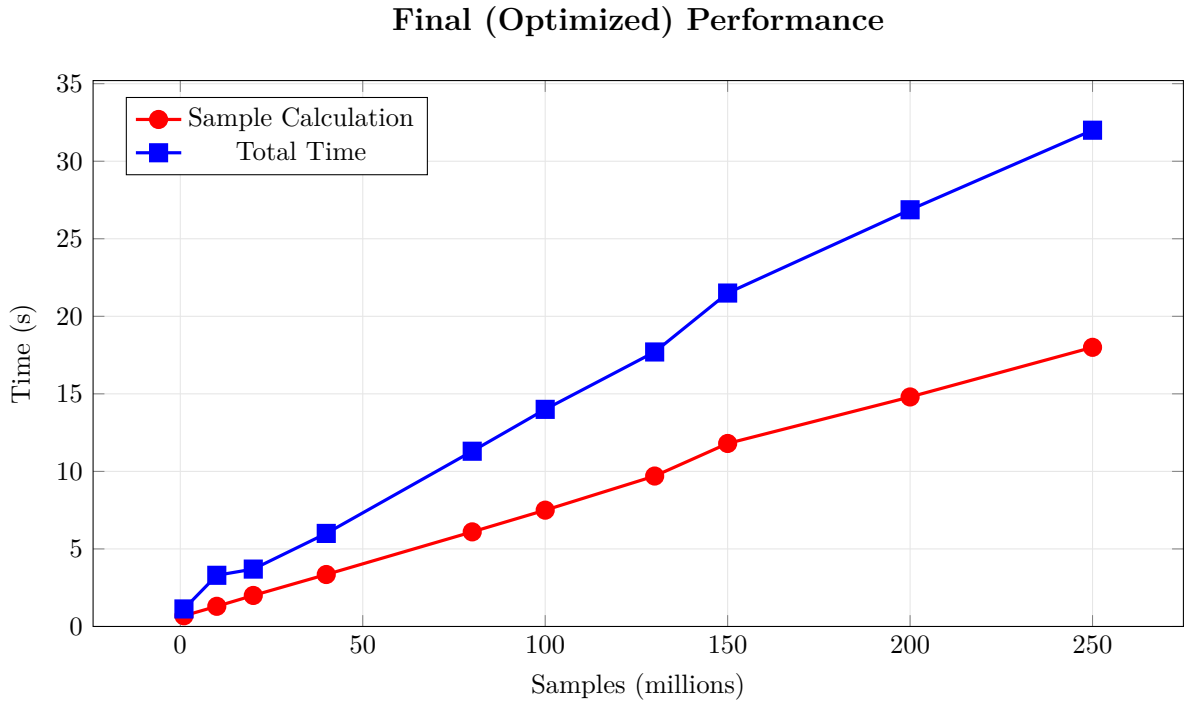


Figure 3.9: Final (optimized) performance.

Despite the fact that roughly half of the total execution time is now spent on sample generation, no further meaningful improvements can be achieved within the Python-based implementation. The waveform computation already relies on fully vectorized NumPy operations, and the observed performance appears to be limited by memory bandwidth rather than algorithmic inefficiency. It is therefore unlikely that additional optimization attempts in Python would yield noticeable benefits. It should be noted, however, that the C-based server used in the Quantum Optics laboratory may be capable of generating samples considerably faster due to lower-level control and reduced overhead, potentially mitigating this bottleneck in that environment.

In summary, the systematic optimization of the waveform-generation pipeline led to a substantial reduction in end-to-end transfer time, transforming the system from an initially slow and heavily bottlenecked implementation into one that operates close to the theoretical limits of the underlying hardware. The most significant gains resulted from improving the Ethernet throughput and the DDR write path on the ARM processor, while the use of vectorized NumPy operations reduced the cost of sample generation as far as realistically possible within the Python environment. After these improvements, the remaining latency is dominated by the waveform computation on the control PC, a limitation that cannot be further addressed without transitioning to a lower-level implementation such as the C-based server used in the laboratory. Overall, the final system achieves an approximately sixfold speedup across all waveform sizes, providing a robust and efficient foundation for high-throughput waveform transfer and playback.

High Precision Internal Triggering

4.1 Previous Trigger Problems

In the previous system design, waveform playback could be initiated either by an external hardware trigger or by an internal trigger generated on the FPGA. When using the internal trigger, however, the system exhibited a delay between successive waveform playbacks that was both unavoidable and non-deterministic.

This behavior originated from the way the internal trigger was generated. The trigger signal would be issued by the ARM processor after the DMA engines had fully completed waveform playback and had been re-initialized. Because the ARM processor runs an operating system and executes multiple tasks in parallel, the exact timing of this trigger varied depending on system load, making precise and repeatable timing impossible.

This effect can be quantitatively demonstrated by the graph shown below.

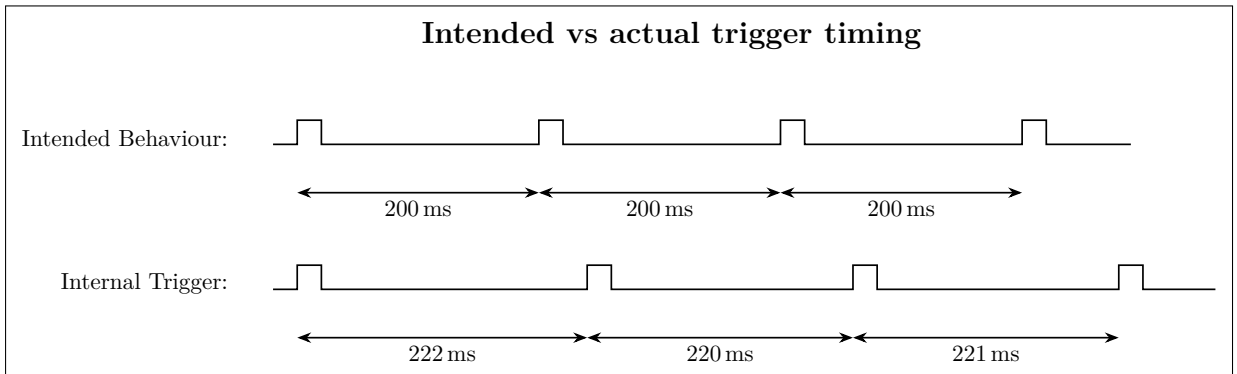


Figure 4.1: Comparison between the intended trigger timing and the actual internal trigger behaviour for a waveform with a period of 200 ms. Ideally, a trigger should be generated every 200 ms; however, the internal trigger exhibits an additional, non-deterministic latency of approximately 20 ms.

Even for very small waveforms (approximately 2 k samples), the system was required to wait more than 20 ms before issuing a new trigger. Under normal operating conditions, particularly

4. HIGH PRECISION INTERNAL TRIGGERING

when larger waveforms were used, the delay between waveform writes frequently exceeded 200 ms. These delays were directly observable on the oscilloscope.

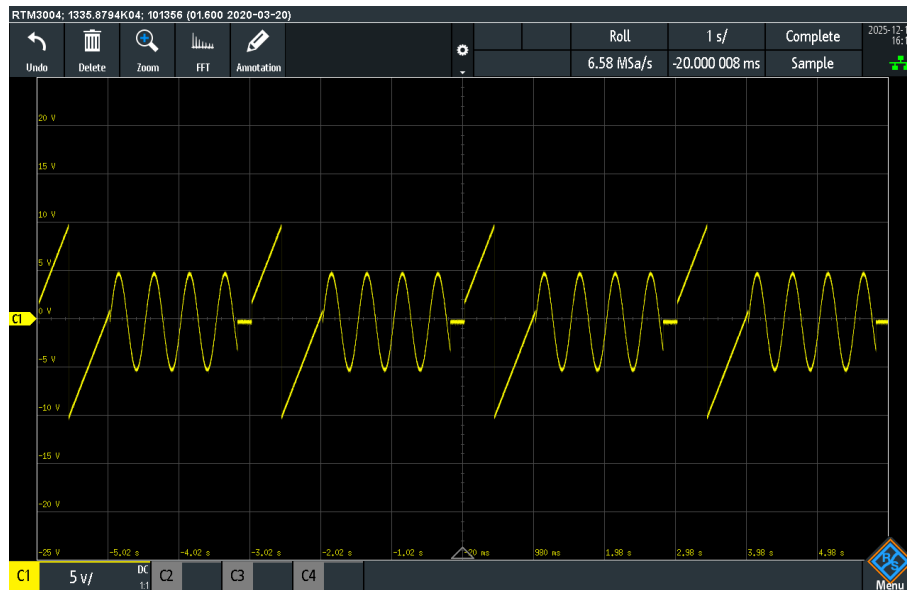


Figure 4.2: Oscilloscope capture showing a waveform composed of a sawtooth wave and a sine wave. After the waveform completes, a short latency (zeros) is observed before playback restarts.

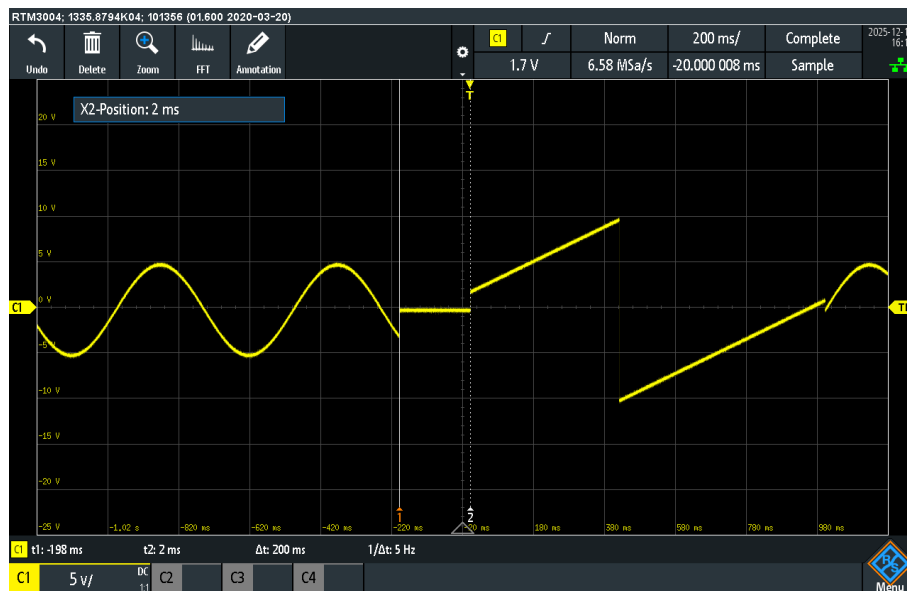


Figure 4.3: Oscilloscope capture showing the same waveform as before. The latency measurement (bottom left) indicates an approximate delay of 200 ms before playback restarts.

Finally, the user had no control over how many trigger pulses would be sent and therefore no way to specify how many times a waveform should be replayed. Because the trigger is the

4. HIGH PRECISION INTERNAL TRIGGERING

signal that instructs the SPI module to begin waveform playback, these timing uncertainties and missing controls directly limited channel synchronization and reduced the reliability of time-critical experimental sequences.

4.2 Implementation

The only way to eliminate these issues and obtain a fully deterministic trigger was to move the entire triggering mechanism into dedicated FPGA logic, ensuring cycle-accurate timing and complete user control.

Therefore, a custom Vivado logic block was designed called `internal_trigger_gen`, whose job is to produce the internal trigger signal in a fully deterministic way, while also allowing the user to set the trigger period and to specify how many triggers—and thus how many waveform repetitions—should be generated.

The `internal_trigger_gen` module receives three main trigger-related inputs: `nonstop`, `period`, and `n_triggers`. These signals define the trigger generation configuration and are provided to the module through the processor slave registers. In addition to these signals, the module also receives several channel-related inputs, namely `max_period_channel` for each channel. These additional inputs are not involved in the internal trigger logic itself; instead, they are simply forwarded to the SPI modules through this block and will be discussed in the final section of the thesis.

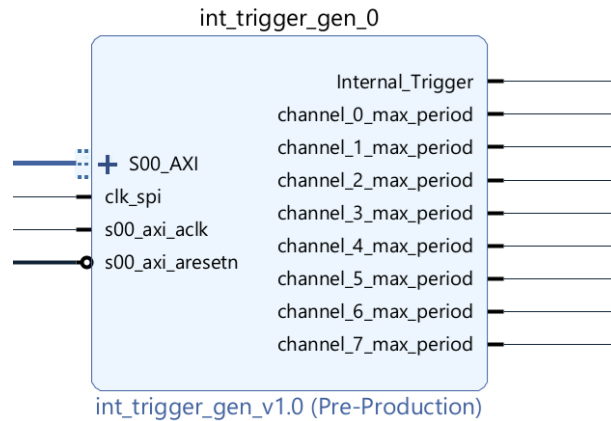


Figure 4.4: The internal trigger module. Inputs are provided via the left-side `S00_AXI` wire connected to the slave registers of the processor.

The trigger-related parameters (`nonstop`, `period`, and `n_triggers`) are provided directly by the user through the Python control software. The user enters these values on the control PC, and they are sent together with the rest of the waveform metadata to the ARM processor. The ARM then writes them into the corresponding slave registers inside the `internal_trigger_gen` module, ensuring that the FPGA logic always has access to the desired trigger configuration.

In particular, the user selects whether the system should run in nonstop mode or in finite-trigger mode by setting the `nonstop` field in the trigger settings interface, as shown in the next Figure:

4. HIGH PRECISION INTERNAL TRIGGERING

```
#  
##  
# Use internal trigger (0) external trigger (1) debug trigger (2) internal nonstop (3)  
trigger_mode = 3
```

Figure 4.5: User interface for selecting the trigger mode. In this configuration, `trigger_mode = 0` corresponds to the internal trigger with a user-defined number of trigger pulses, while `trigger_mode = 3` enables the internal trigger in nonstop mode.

For the other two parameters, `period` and `n_triggers`, the user provides them as part of the waveform parameters that are sent to the ARM processor. The `period` value specifies the time between two internal triggers, expressed in units of the FPGA clock, where each clock tick corresponds to 8 ns. Since entering such small time units directly would be impractical, the Python interface allows the user to specify the period in microseconds. This value is then internally converted into FPGA ticks by multiplying it by 125, ensuring that the hardware receives the correct 8 ns-based period. The module supports a 40-bit period value, giving a maximum delay of

$$(2^{40} - 1) \times 8 \text{ ns} \approx 8796 \text{ s} \approx 146.6 \text{ min} \approx 2.44 \text{ hours.}$$

The `n_triggers` parameter is a 32-bit value, allowing up to

$$2^{32} - 1 = 4294967295$$

individual trigger pulses in finite-trigger mode.

In the Python interface these values are written into the variables shown in the following Figure, and the ARM forwards them to the corresponding slave registers inside the `internal_trigger_gen` module.

```
#To be inputed by the user,Period in microseconds  
period=3000000  
#n_triggers is the number of waveforms before we stop,n_triggers has to be < 2^31  
n_triggers=1
```

Figure 4.6: Python interface variables where the user specifies the `period` and `n_triggers` parameters.

Internally, the `internal_trigger_gen` module operates using a simple counter-based mechanism. A main counter continuously measures the time elapsed since the last trigger was issued, and whenever this counter reaches the user-defined `period` value, the module generates a new trigger pulse and resets the counter. To avoid unnecessary latency, the module also produces an initial trigger immediately when it is enabled, so that waveform playback starts at once rather than waiting for the first period to elapse—a delay that could otherwise be extremely long, since the allowed period range exceeds two hours. In finite-trigger mode (`nonstop = 0`), a second counter is activated to keep track of how many triggers have already been generated. Once this counter reaches the user-specified `n_triggers` value, the module asserts a signal called `end_triggers`, which disables further trigger generation and cleanly terminates the playback sequence. Since the triggers are the signals that start each waveform, no additional waveforms are played after `end_triggers` is asserted.

4. HIGH PRECISION INTERNAL TRIGGERING

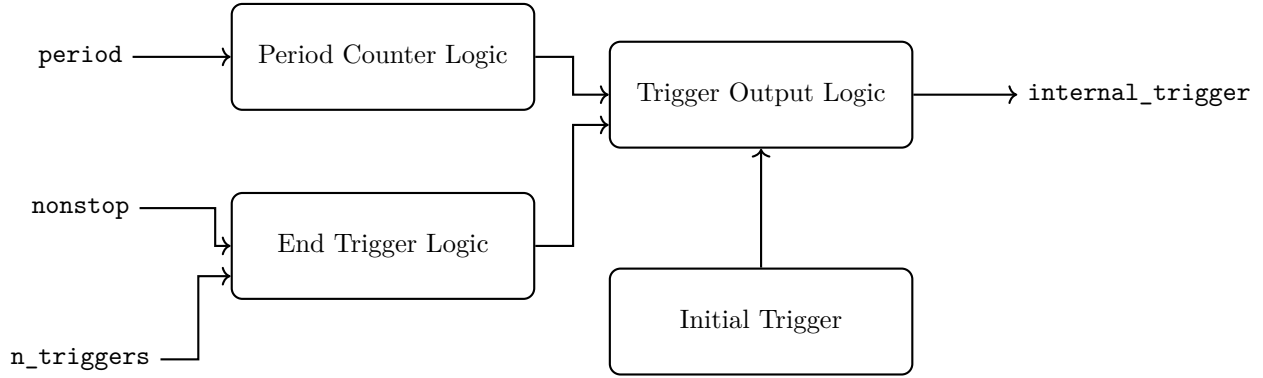


Figure 4.7: Logic structure of the `internal_trigger_gen` module. The outputs of the Period Counter, End Trigger Logic, and Initial Trigger converge into a single Trigger Output Logic block, which generates the final `internal_trigger` signal.

4.3 Results

The introduction of the `internal_trigger_gen` module successfully resolved the two main limitations of the previous triggering approach. First, by moving trigger generation entirely into FPGA logic, internal triggers are now produced with fully deterministic, cycle-accurate timing at a resolution of one FPGA clock cycle (8 ns), independent of processor load or operating system scheduling. Second, the user gained explicit control over the number of waveform repetitions through the `n_triggers` parameter, eliminating the previous lack of control over trigger count. As shown in the figure below, the trigger-counting mechanism reliably terminates waveform playback once the user-defined `n_triggers` value is reached. Together, these improvements enable reliable channel synchronization and predictable timing behavior for time-critical experimental sequences.

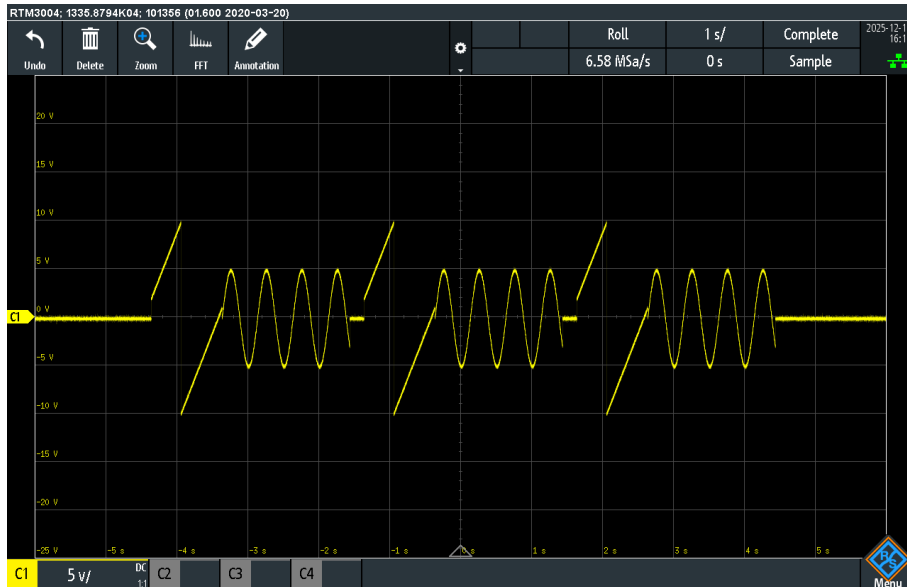


Figure 4.8: Illustration of trigger counting in finite-trigger mode. In this example, the user sets `n_triggers` = 3, resulting in three waveform playbacks before waveform generation terminates.

4. HIGH PRECISION INTERNAL TRIGGERING

However, while trigger determinism was fully resolved, the system still could not replay waveforms back-to-back without interruption. Even with the internal trigger mechanism in place, a mandatory pause between successive waveform playbacks persisted due to remaining processor-side overhead.

Consequently, while the internal triggering mechanism itself was successfully resolved, the overall waveform playback latency remained a limiting factor for certain high-speed applications. Addressing this remaining overhead required additional architectural considerations beyond trigger generation alone.

Continuous Playback

5.1 Limitations of Previous Control

Although the FPGA-based internal trigger generator provided fully deterministic trigger timing, the system was still unable to achieve continuous, back-to-back waveform playback. Instead, delays occurred between successive waveform segments, preventing uninterrupted output.

This behavior is illustrated in the diagram below by the discontinuities between successive waveforms.

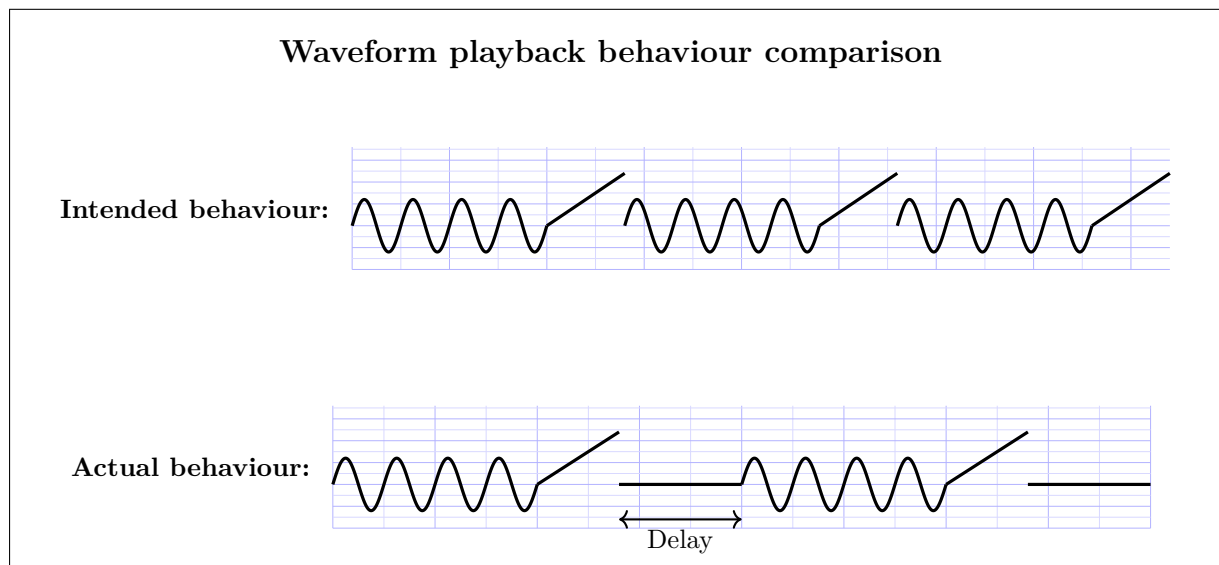


Figure 5.1: Waveform playback behaviour comparison. The waveform is made up of a sine wave followed by a ramp-up segment. In the intended behaviour (top), these segments are played one after another without interruption, resulting in continuous output. In the actual behaviour (bottom), delays occur between waveform segments. These delays appear as flat, zero-valued regions, showing that the waveform playback is not continuous.

5. CONTINUOUS PLAYBACK

This limitation was no longer related to trigger generation, but rather to the control architecture governing DMA operation and playback sequencing. In particular, two fundamental architectural limitations persisted.

Firstly, the DMA engines were entirely controlled by the ARM processor. For each waveform playback, the processor explicitly issued a start command to the DMAs, which then transferred a finite block of waveform samples into the FIFOs feeding the SPI modules. Once the programmed transfer completed, the DMAs stopped issuing data and remained idle. No further samples were sent unless the processor reconfigured and restarted the DMAs. As a result, the FIFOs eventually became empty, leaving the SPI modules without data to transmit. Even with internally generated triggers, continuous playback was therefore impossible because no new samples were available after the initial transfer.

Second, and independently of the DMA behavior, the existing FPGA state machine was not equipped to handle the arrival of new data after a playback had completed. The control logic lacked defined states and transitions to rearm the data path, and resume playback at the end of a waveform without processor involvement. More specifically there was no mechanism to correctly accept renewed data from the DMAs after waveform end. Consequently, even if continuous data delivery had been available, the state machine itself could not have sustained uninterrupted playback.

Together, these two limitations confined the system to segmented, single-shot operation and required processor intervention between waveform playbacks. Overcoming both issues required reworking the DMA strategy and redesigning the FPGA state machine to autonomously manage data flow and playback sequencing.

5.2 Cyclic Mode DMAs

The first requirement for achieving continuous waveform playback was ensuring uninterrupted data availability at the input of the SPI modules. As long as the DMA engines were unable to continuously provide samples, the FIFOs feeding the SPI modules would eventually become empty, preventing correct waveform reproduction. Therefore, the DMA engines had to be configured such that they were always ready to supply new data, without requiring intervention from the ARM processor.

In particular, once a waveform had finished playing, the DMA engines needed to be immediately prepared to retransmit the same waveform data to the FIFOs. Any pause caused by software reconfiguration or DMA restart would reintroduce gaps in playback and defeat the purpose of continuous operation.

This requirement is fulfilled by operating the DMA engines in scatter-gather cyclic mode. In this mode, waveform data is described by a linked list of DMA descriptors that form a closed loop. After the last descriptor has been processed, the DMA engine automatically wraps the pointer back to the first descriptor, thereby restarting the transfer from the beginning of the waveform without interruption as shown in the figure below.

5. CONTINUOUS PLAYBACK

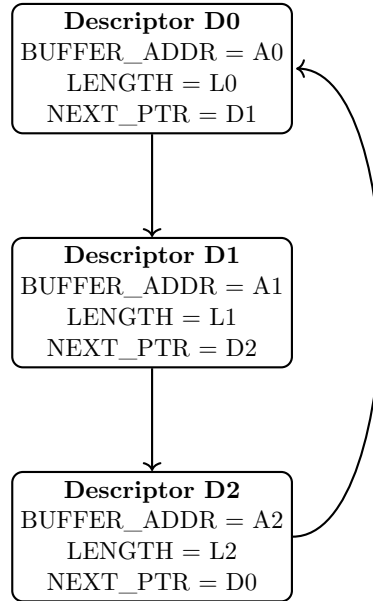


Figure 5.2: Scatter-gather DMA descriptor structure configured in cyclic mode. After the last descriptor has been processed, its `NEXT_PTR` points back to the first descriptor, causing the DMA engine to restart the transfer from the beginning of the waveform and enabling continuous, uninterrupted playback.

As a result, waveform samples are continuously streamed to the FIFOs, enabling seamless waveform replay and ensuring that the SPI modules always have valid data to transmit.

To implement this behavior, the following steps are required. First, the DMA driver creates the descriptor ring and submits it to the hardware. After this initialization step, cyclic operation is enabled by configuring the descriptor ring with `XAxiDma_BdRingEnableCyclicDMA()`, which allows the last descriptor in the ring to wrap back to the first. Subsequently, cyclic mode is selected in the DMA engine itself for the transmit direction using `XAxiDma_SelectCyclicMode()`. Only after both configurations have been applied can the DMA be safely started in cyclic operation.

5. CONTINUOUS PLAYBACK

```

/* Submit to hardware */
Status = XAxiDma_BdRingToHw(TxRingPtr, NumBds, BdPtr);
if (Status != XST_SUCCESS) {
    xil_printf("DMA %d: BD submission failed\r\n", i_dma);
    return XST_FAILURE;
}

/* Enable cyclic DMA mode for this ring */
XAxiDma_BdRingEnableCyclicDMA(TxRingPtr);
/* Also select cyclic mode in the DMA engine (TX direction) */
Status = XAxiDma_SelectCyclicMode(DmaInst, XAXIDMA_DMA_TO_DEVICE, TRUE);
if (Status != XST_SUCCESS) {
    xil_printf("DMA %d: SelectCyclicMode failed\r\n", i_dma);
    return XST_FAILURE;
}

/* Start DMA hardware */
Status = XAxiDma_BdRingStart(TxRingPtr);
if (Status != XST_SUCCESS) {
    xil_printf("DMA %d: BdRingStart failed\r\n", i_dma);
    return XST_FAILURE;
}

```

Figure 5.3: First, after the scatter-gather descriptors have been created and linked to form a ring, they are submitted to the DMA hardware. Second, cyclic mode is enabled at both the descriptor ring level and the DMA engine level. Finally, the DMA transfer is started, allowing continuous waveform streaming.

Configuring the DMA engines in cyclic scatter-gather mode enables fully autonomous operation without processor intervention. Once started, the DMA continuously streams waveform data without interruption, ensuring that the FIFOs feeding the SPI modules never become empty. This behavior was verified using an Integrated Logic Analyzer (ILA) core, which confirmed correct and uninterrupted data flow from the DMA to the FIFOs, allowing the SPI modules to reliably receive and transmit waveform samples.

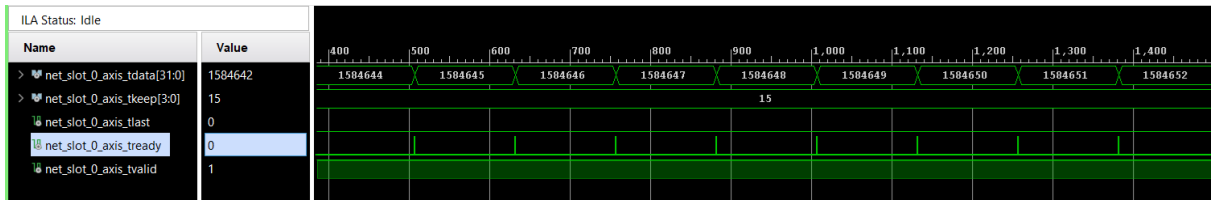


Figure 5.4: ILA capture demonstrating continuous DMA data streaming across waveform boundaries. The `tvalid` signal remains asserted, indicating uninterrupted data transfer, while `tdata` updates every 125 clock cycles, matching the required sample rate for SPI waveform playback.

5.3 New State Machine Architecture

The final component that required modification in order to enable continuous waveform playback was the control state machine within the SPI transmission modules. Before describing the changes introduced in the new architecture, it is first necessary to outline the operation of the original state machine and to highlight how its structure inherently limited continuous playback.

The original finite state machine was organized into two distinct operational phases: a DAC initialization phase and a triggered waveform playback phase. Both phases were designed for finite, event-driven operation and relied on explicit control flow and trigger events.

5. CONTINUOUS PLAYBACK

In the DAC initialization phase, the state machine starts in the `INIT` state after reset and remains idle until valid AXI-stream data and a start pulse are detected. It then transitions to `FETCH_DAC_CONFIG_DATA` to retrieve the DAC configuration parameters and the initial output value from BRAM. These values are prepared and transmitted to the DAC in the `PREP_DAC_INIT_VAL` and `PREP_DAC_CONFIG` states. SPI communication is handled by the `SPI_BOOT` and `SPI_SEND` states, while `UPDATE_DAC_REG_AND_OUTPUT` latches the transmitted data into the DAC. Once DAC initialization and configuration are complete, the state machine exits this phase and transitions to the trigger-handling logic.

Phase 1: DAC Initialization

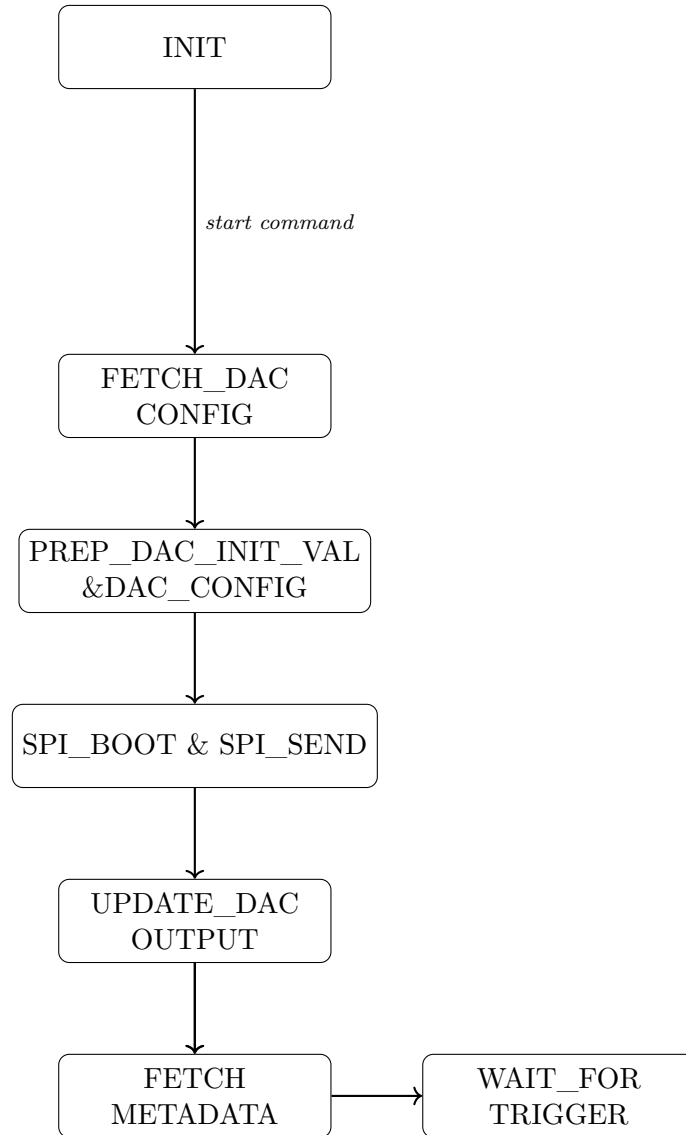


Figure 5.5: Simplification of the state machine for DAC initialization and configuration. After the DAC initial value and configuration data have been successfully transmitted, the initialization loop terminates and control transitions to the `WAIT_FOR_TRIGGER` state, which marks the handover to the waveform playback phase of the state machine.

5. CONTINUOUS PLAYBACK

In the triggered waveform playback phase, the state machine waits in the `WAIT_FOR_TRIG` state until a trigger pulse is received. Upon triggering, playback starts by briefly entering the `UPDATE_DAC_REG_AND_OUTPUT` state, after which waveform samples are prepared in the `PREP_DAC_NEXT_SAMPLE` state and transmitted during the `SPI_BOOT` and `SPI_SEND` states.

Each sample is transmitted using the same SPI transfer sequence as in the initialization phase. Playback proceeds in a finite loop until the programmed number of samples of our waveform has been exhausted, after which the state machine returns to the `INIT` state, waiting for a new start command from the arm processor.

Old State Machine Phase 2: Triggered Playback

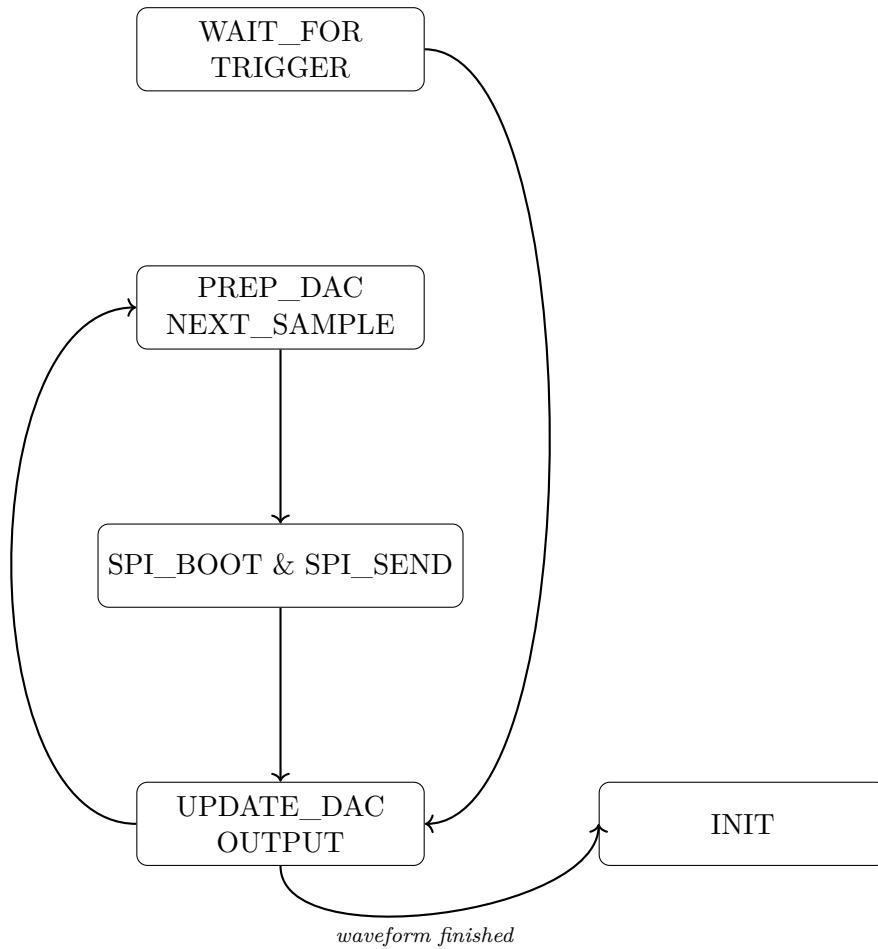


Figure 5.6: Simplification of the initial triggered waveform playback state machine. After a trigger event, samples are transmitted to the DAC in a finite loop. Once all programmed samples and events have been played, the state machine exits the playback loop and returns to the `INIT` state, re-entering the DAC initialization phase before the next trigger.

In the original design, the playback phase always returns to the DAC initialization phase, preventing continuous waveform streaming. While suitable for finite, trigger-based operation, this behavior becomes unnecessary and inefficient once continuous DMA data delivery is avail-

5. CONTINUOUS PLAYBACK

able, as repeated reinitialization introduces avoidable overhead.

To enable continuous waveform playback, it was therefore necessary to modify the control flow such that the second phase of the state machine—the triggered playback phase—could operate independently of the initialization phase. Instead of resetting to `INIT` after the end of a waveform, the state machine must remain in a ready state, waiting for the next trigger event. Upon re-triggering, waveform playback should restart immediately using the already configured DAC, without repeating the initialization and configuration sequence.

Therefore, the solution was to introduce an additional control state, denoted `REPEAT_PREP`, whose purpose is to decouple waveform replay from the DAC initialization phase. In the original control flow, completion of waveform playback caused the state machine to return to `INIT`. In the revised architecture, this transition is replaced by a transition to `REPEAT_PREP`.

This new state prepares the control logic for waveform repetition without reinitializing the DAC. Specifically, it re-arms the playback path and transitions the state machine directly to `FETCH_METADATA`, allowing waveform playback to restart immediately upon the next trigger event. In this way, successive waveform repetitions can be executed back-to-back using the already configured DAC. The `REPEAT_PREP` state also incorporates additional control logic to ensure correct sequencing and synchronization, which will be discussed in detail in the following sections.

Phase 2 Changes: Triggered Playback

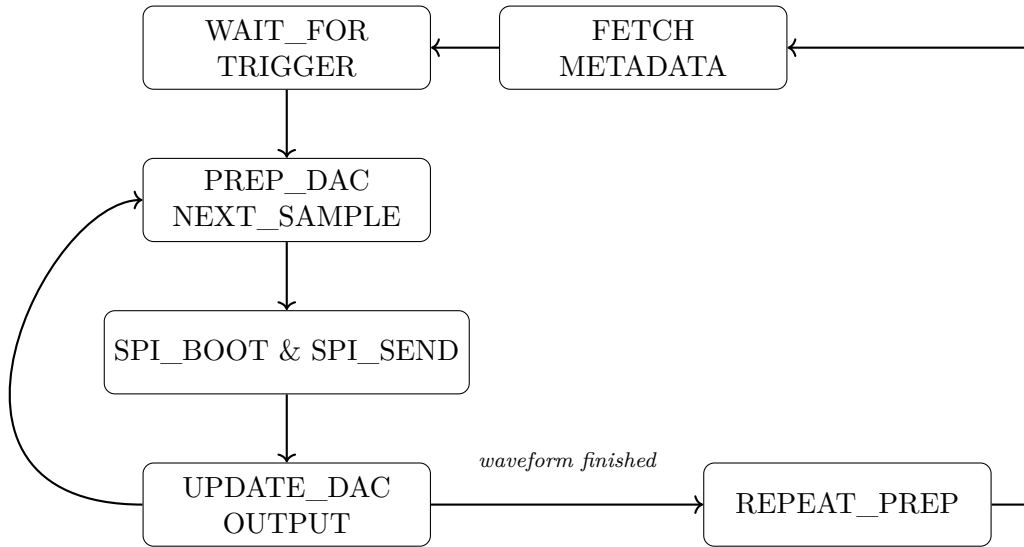


Figure 5.7: Simplified high-level control flow of the triggered waveform playback state machine with the added `REPEAT_PREP` state. Playback follows the original sequence until completion, after which control transitions to `REPEAT_PREP` and returns to `WAIT_FOR_TRIG`, allowing the waveform to be replayed on the next trigger without reinitializing the DAC or waiting for a processor start signal.

5. CONTINUOUS PLAYBACK

5.4 Implementation Details

The waveform generator supports eight output channels that must remain synchronized, even when individual channels are assigned waveforms with different durations. Channels that complete playback earlier than others must wait for the next common trigger event while outputting a defined idle value (zero) to preserve correct and stable output behavior.

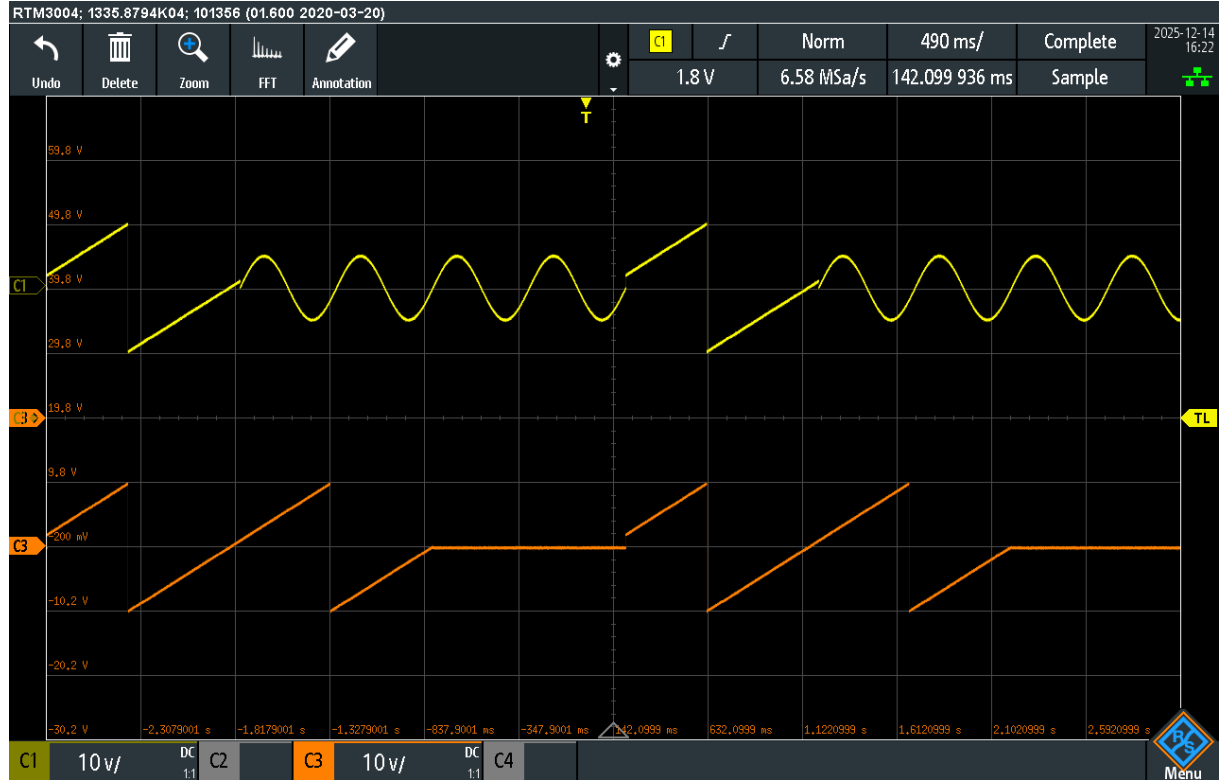


Figure 5.8: Oscilloscope capture showing waveform channels with different periods and their intended playback operation. The yellow trace (Channel 1) consists of a sawtooth waveform and a sine waveform and has the largest period; therefore, it determines the playback completion point and must restart immediately on the next trigger. The orange trace (Channel 2) consists of a sawtooth waveform with a shorter period and is expected to wait until Channel 1 completes before restarting, outputting zeros in the interim.

Before waveform playback begins, the ARM processor determines which output channels have the longest waveform period. This is done using the `compute_period_channels_bitmask` function, which compares the configured periods of all eight channels and identifies the channel or channels whose period matches the user-defined maximum. The result is a bitmask in which each bit corresponds to a channel: bits set to 1 indicate channels with the maximum period, while bits set to 0 indicate channels with shorter periods.

5. CONTINUOUS PLAYBACK

```
2090
2091 uint32_t compute_period_channels_bitmask(uint64_t input_period)
2092 {
2093     uint32_t bitmask = 0;
2094
2095     if (input_period == 0) {
2096         return 0;
2097     }
2098
2099     uint64_t period_cycles = input_period / 125;
2100
2101     for (u8_t i = 0; i < g_metadata.num_channels; i++) {
2102         uint64_t channel_period = g_metadata.channels[i].tot_duration;
2103
2104         if (channel_period == period_cycles) {
2105             uint8_t hw_ch = g_metadata.channels[i].ihw_channel;
2106             bitmask |= (1U << hw_ch);
2107         }
2108     }
2109
2110     return bitmask;
2111 }
2112 }
```

Figure 5.9: ARM-side implementation of `compute_period_channels_bitmask`. The function identifies channels with the maximum waveform period and encodes them as a bitmask, which is provided to the FPGA via `channel_max_period` for channel-specific control.

This bitmask is then provided to the FPGA and made available to the control logic through the `channel_max_period` signal mentioned in the previous chapter. The `REPEAT_PREP` state uses this signal to determine the appropriate control flow for each channel once waveform playback has completed.

Channels flagged as having the largest period transition directly to `FETCH_METADATA` (as shown in the previous chapter), allowing immediate preparation for the next waveform and enabling playback to restart with the first sample upon the next trigger event.

Channels with shorter periods instead follow a different control path. After entering the `DAC_SEND_INIT_VAL` state, they proceed through the SPI transmission sequence, including the `SPI_BOOT` and `SPI_SEND` states, and then `UPDATE_DAC_REG_AND_OUTPUT` state to transmit a defined idle value. Once the idle value has been applied, the state machine returns to `REPEAT_PREP` and from there, it transitions to `FETCH_METADATA` and subsequently to the `WAIT_FOR_TRIG` state. Through this mechanism, all channels remain synchronized and correctly aligned for subsequent playback cycles despite differences in waveform duration.

Stage 1: Idle Value Transmission

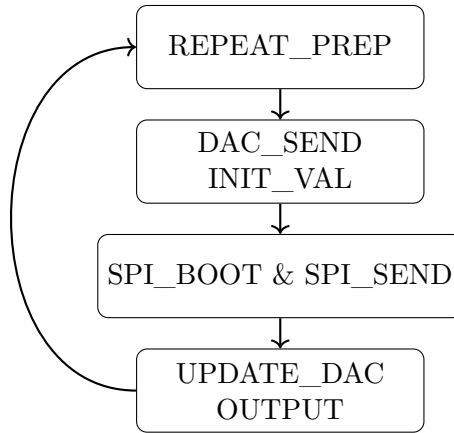


Figure 5.10: Shorter-period channel behavior immediately after waveform completion. Upon entering `REPEAT_PREP`, the state machine transitions through the DAC SPI sequence to output a defined idle value. After updating the DAC output, control returns to `REPEAT_PREP`. From this point onward, the state machine enters the second stage of operation.

Stage 2: Preparation for Next Trigger

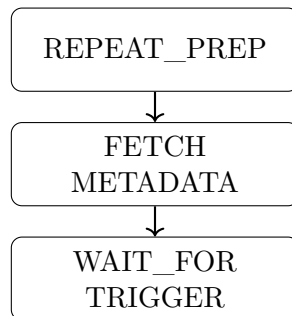


Figure 5.11: Simplified version of the second-stage control flow for shorter-period channels: After returning to `REPEAT_PREP`, the state machine follows the normal playback preparation path, including metadata fetch and DAC register update, before transitioning to `WAIT_FOR_TRIG` to await the next global trigger event.

5.5 Results

The implemented continuous playback architecture was experimentally evaluated using multiple output channels configured with different waveform periods (both maximum-period and shorter-period waveform configurations). In all tested cases, the waveform generator operated as expected, maintaining correct timing, data integrity, and channel synchronization without requiring processor intervention between playback cycles.

5. CONTINUOUS PLAYBACK

For channels configured with the largest waveform period, playback restarts immediately after waveform completion. In this case, the first sample of the subsequent waveform segment is observed directly after the last sample of the previous segment, confirming uninterrupted data delivery and correct state machine operation. In contrast, channels with shorter waveform periods wait until the maximum-period channel has completed before restarting (as shown in Fig. 5.8) , ensuring synchronized and correct playback across all channels.

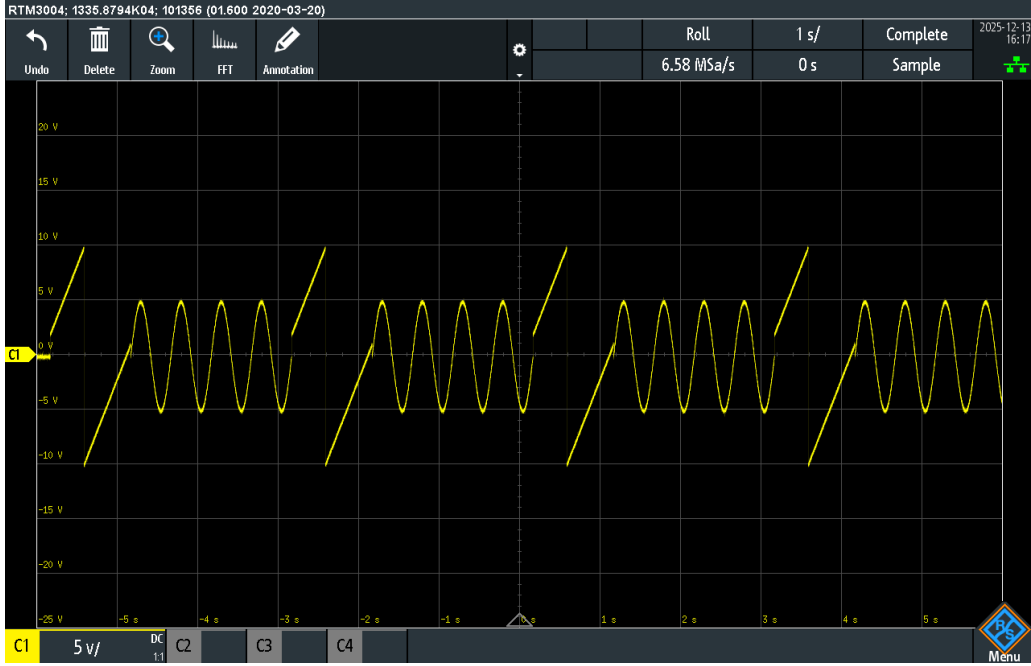


Figure 5.12: Oscilloscope capture of the same signal described above (sawtooth wave & sine wave), showing no latency between successive waveform repetitions.

It was further verified that waveform samples are transmitted at a fixed interval of 125 clock cycles, matching the DAC sampling requirement of $1\text{ }\mu\text{s}$ ($125 \times 8\text{ ns}$). This behavior was confirmed using an Integrated Logic Analyzer (ILA) core, which showed stable and deterministic sample timing across extended playback. The `ss_no` signal was used to observe, in real time, when samples were written to the DAC register, ensuring that each sample was delivered exactly when expected.

5. CONTINUOUS PLAYBACK

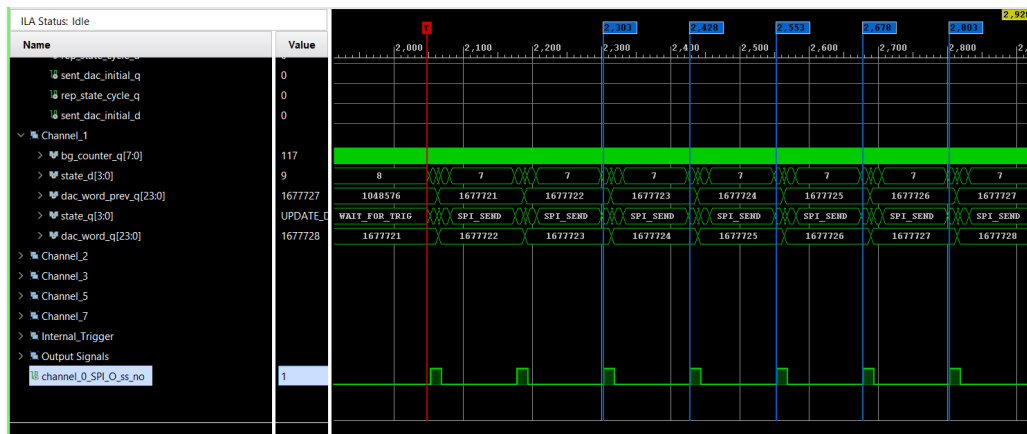


Figure 5.13: ILA capture showing the distance between consecutive `ss_no` activations. Each activation occurs exactly 125 clock cycles apart, corresponding to the 1 μ s DAC sampling period. A new waveform sample is transmitted on every activation, as indicated by the update of the `dac_word` signal.

This timing behavior was validated across all relevant edge cases, including triggers occurring immediately between successive waveforms and waveforms consisting of a single sample. In all cases, samples continued to be transmitted at the correct $1\,\mu\text{s}$ interval, confirming robust and stable operation of the continuous playback architecture.

Moreover, for the most important case at waveform boundaries, oscilloscope measurements confirmed that the signal transitions directly from the last sample of one waveform to the first sample of the next, with no intervening idle (zero-valued) samples, ensuring truly continuous playback.



Figure 5.14: Oscilloscope capture of the output waveform around a waveform boundary, shown to provide context for the transition between successive waveform segments.

5. CONTINUOUS PLAYBACK

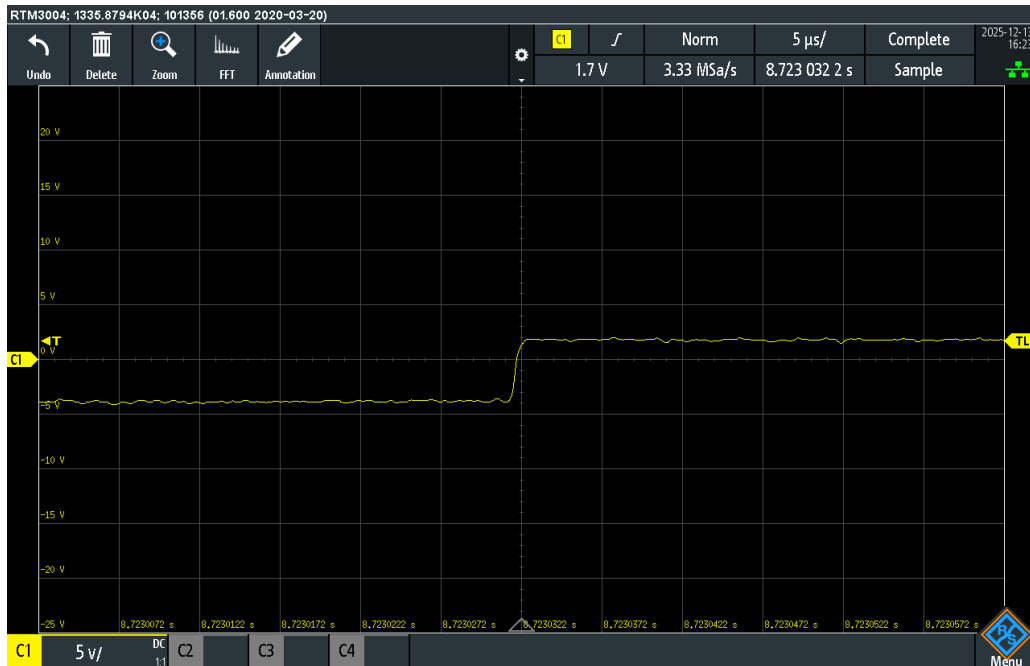


Figure 5.15: Oscilloscope capture of the same waveform as in the previous figure, shown at a $5\mu\text{s}$ time scale, demonstrating that the signal transitions directly from the last sample of one waveform to the first sample of the next, with a direct step from approximately -4 V to 2.5 V , with no intermediate or idle (zero-valued) samples.

Additionally, synchronization across channels was also tested using different Ethernet cable lengths between the FPGA and the DAC boards, with one channel connected using a 1,m cable and the other using a 10,m cable. The same waveform was applied to both channels, and oscilloscope measurements showed that the two signals overlapped in time, confirming that the channels are exactly synchronized despite the difference in cable length.

5. CONTINUOUS PLAYBACK

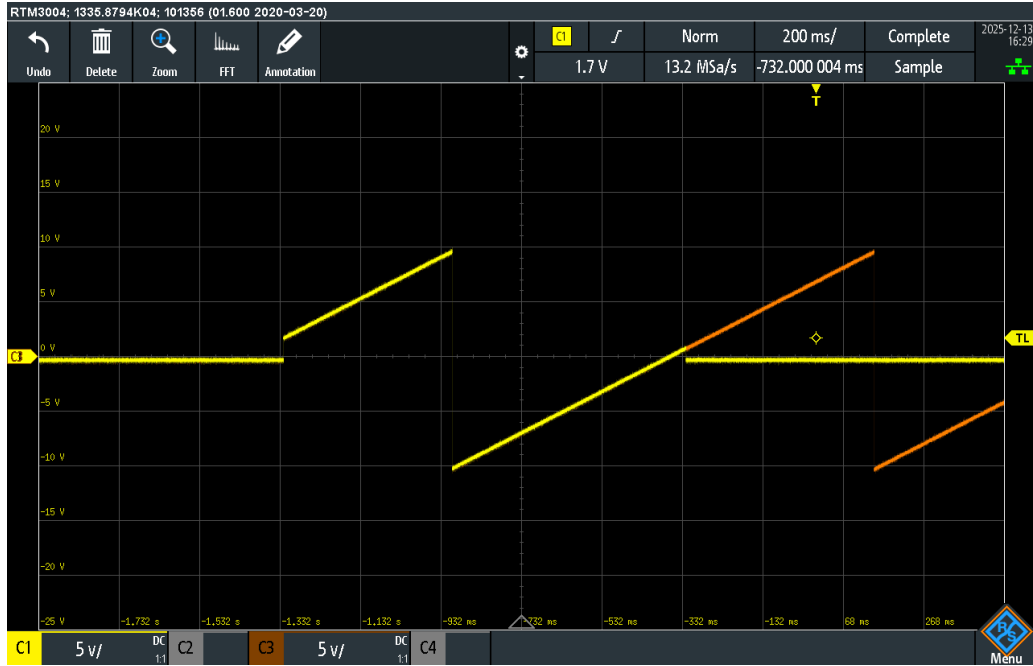


Figure 5.16: Oscilloscope capture demonstrating channel synchronization. The orange trace (Channel 1) and the yellow trace (Channel 2) are both sawtooth waveforms, with Channel 1 configured with a larger period. During the initial portion of the waveform, before Channel 2 completes playback, the two signals overlap exactly. The orange trace is not visible in this region because it is completely hidden behind the yellow trace, indicating that both channels have identical values at every sample. This behavior confirms exact synchronization between the channels.

Finally, several additional edge cases were examined to ensure robust operation of the re-designed state machine. Where necessary, small refinements were made to the control logic to maintain compatibility with continuous playback. The tested and identified edge cases include the following:

- Verifying correct operation of all channels by selectively enabling individual channels and channel combinations.
- Playing a single-sample waveform across all channels.
- Playing a single-sample waveform on a single channel only.
- Configuring multiple waveform events for a channel, each consisting of a single sample.
- Operating with an external trigger period equal to the waveform length,
- Including behavior under `immediate_stop` and `stop_after_waveform_end` commands.

Conclusion

6.1 Summary of Improvements and Impact on Quantum Experiments

This thesis improved an FPGA-based eight-channel arbitrary waveform generator for quantum gas experiments by addressing key limitations in memory usage, data transfer speed, trigger determinism, and waveform playback continuity. Memory utilization was expanded using scatter-gather DMA, waveform upload speed was significantly increased by optimizing waveform generation and Ethernet transfer, deterministic internal triggering was implemented in FPGA logic, and continuous back-to-back waveform playback was enabled through a redesigned DMA and control state machine.

The improvements presented in this thesis enhance both the usability and reliability of the waveform generator for quantum gas experiments. By expanding the available waveform memory and reducing waveform upload times, the system supports larger and more complex experimental sequences while minimizing preparation overhead.

In addition, deterministic internal triggering and continuous waveform playback provide precise timing control and stable, uninterrupted analog outputs across all channels. These capabilities are essential for experiments requiring tight synchronization, long-duration operation, or rapid waveform updates, making the upgraded system a flexible and robust control platform for future experimental needs.

6.2 Implementation Notes and Future Considerations

Implementation Notes

- The first output channel must always be configured with the maximum waveform period and remain active, as it defines the common clocks for all channels. This requirement follows the original design assumptions described in Bahadir’s thesis [1] and ensures correct synchronization across channels.
- In the external trigger case, continuous playback is not supported, even for the channel with the largest waveform period. Because of this limitation, an idle output value must be applied for at least $1\ \mu\text{s}$ between successive waveform playbacks. As a result, the external trigger period must be set at least $1\ \mu\text{s}$ longer than the waveform duration to allow safe replay to restart.

6. CONCLUSION

- To ensure stable operation and avoid undefined behavior, the user-defined internal trigger period must be greater than or equal to the longest waveform period across all channels. If a shorter period is configured, the system may enter an undefined state. In such cases, it is recommended to power-cycle the FPGA before resuming operation.
- Additional validation checks were added to the Python control interface to verify waveform consistency before transmission. These checks ensure that the total number of samples does not exceed the FPGA limit (250 million), and that the actual waveform length matches the metadata configuration, preventing invalid or inconsistent playback configurations.
- Two TCP stop control flags are supported: When sent to the FPGA `tcp_data_stop` allows the current waveform to complete before stopping playback, and `tcp_data_stop_immediately`, halts waveform playback immediately without waiting for waveform completion.
- During testing, one of the DAC boards exhibited unexpected and unstable behavior. This note is included for future engineers working on the project to be aware of this issue. Potential causes include a loosely connected SMA-to-BNC adapter or insufficient mechanical contact between the upper and lower DAC boards. In some instances, output voltages as low as -13 V were observed, despite the intended operating range being limited to -10 V to $+10\text{ V}$. Careful inspection and secure mechanical assembly are therefore strongly recommended.

Future Considerations

- For further performance improvements, the control software could be migrated from the current Python-based implementation to a C or C++ environment. A lower-level implementation may reduce waveform generation and data-handling overhead, particularly for large waveform datasets used in laboratory operation.
- Furthermore, the whole setup was also tested with the C++-based GUI (Matrix) used by the quantum optics group; the GUI functioned correctly for some test cases, but further testing is required to fully confirm its reliability and performance across all use cases.
- The current hot-plugging limitation of the Ethernet connections between the FPGA box and the DAC boards should be addressed. At present, connecting Ethernet cables while the 24 V supply is connected can damage the hardware.; resolving this issue is essential to improve system robustness and operational safety.
- Opening and closing the FPGA box is required every time the flash memory needs to be updated and is challenging due to the large amount of internal cabling. Using a new printed circuit board (PCB) that simplifies internal connections would significantly improve accessibility, reduce handling complexity, and minimize the risk of wiring errors.

6.3 Final Notes

This thesis was carried out as part of ongoing development within the Quantum Optics group and builds directly upon previous work. The implemented changes aim to improve reliability and usability rather than introduce a complete redesign, and are intended to support future experimental work.

Bibliography

- [1] Bahadır Dönmez. “Towards a Distributed Low-Noise Signal Generation System for Quantum Gas Experiments: A Scalable FPGA-Based Design”. Master’s Thesis. ETH Zürich, 2022.
- [2] Shijia Chen. “A Low-Noise Eight-Channel Arbitrary Waveform Generator for Quantum Gas Experiments”. Master’s Thesis. Quantum Optics Laboratory, Sept. 2023.
- [3] Kevin Mondoloni. “Characterisation and Optimisation of a Custom AWG for Low-Noise and High Precision Control of Quantum Gas Experiments”. Semester Thesis. Quantum Optics Laboratory, Aug. 2023.
- [4] Marvin Schwander and Julian Caredda. *Ground Loop Characterisation and Noise Reduction for an FPGA-Based Signal Generator*. Group Project Report. ETH Zürich, Oct. 2024.



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. In consultation with the supervisor, one of the following three options must be selected:

- ☐ I confirm that I authored the work in question independently and in my own words, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies¹.
- ☐ I confirm that I authored the work in question independently and in my own words, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used and cited generative artificial intelligence technologies².
- ☒ I confirm that I authored the work in question independently and in my own words, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used generative artificial intelligence technologies³. In consultation with the supervisor, I did not cite them.

Title of paper or thesis:

FPGA Enhancements in Speed, Memory, and Control for a High-Precision Arbitrary Waveform Generator in Quantum Experiments

Authored by:

If the work was compiled in a group, the names of all authors are required.

Last name(s):

Lydakis

First name(s):

Achilleas Sofoklis

With my signature I confirm the following:

- I have adhered to the rules set out in the Citation Guide.
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date

16/12/2025, Zurich, Switzerland

Signature(s)



If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

¹ E.g. ChatGPT, DALL E 2, Google Bard

² E.g. ChatGPT, DALL E 2, Google Bard

³ E.g. ChatGPT, DALL E 2, Google Bard